

Cryptograph - Lab 1

2.1 The Caesar Cipher

Exercises

1. Manual Decryption: Decrypt the ciphertext `WTAAD` using a shift of `k = 15`
2. Brute Force: You intercept the ciphertext `XAB`. Try all 25 possible shifts to find the English word.
3. Implement a Python function to encrypt and decrypt messages using the Caesar cipher.

Solution to Exercises

Exercise 1 (manual Decryption):

W = 22, T = 19, A = 0, A = 0, D = 3

Applying decryption formula $D(x) = (k - 15) \bmod 26$:

- $(22 - 15) \bmod 26 = 7$ (H)
- $(19 - 15) \bmod 26 = 4$ (E)
- $(0 - 15 = -15 + 26) \bmod 26 = 11$ (L)
- $(0 - 15 = -15 + 26) \bmod 26 = 11$ (L)
- $(3 - 15 = -12 + 26) \bmod 26 = 14$ (O)

Result: HELLO

Exercise 2 (Brute Force):

Formula:

$$D(x) = (x - k) \bmod 26$$

Alphabet:

A=0 B=1 C=2 D=3 E=4 F=5 G=6
H=7 I=8 J=9 K=10 L=11 M=12 N=13
O=14 P=15 Q=16 R=17 S=18 T=19 U=20
V=21 W=22 X=23 Y=24 Z=25

Shift 1

$X = 23 \rightarrow 23 - 1 = 22 \rightarrow W$
 $A = 0 \rightarrow 0 - 1 = -1 \rightarrow 25 \rightarrow Z$
 $B = 1 \rightarrow 1 - 1 = 0 \rightarrow A$
Result: WZA

Shift 2

$X = 23 \rightarrow 23 - 2 = 21 \rightarrow V$
 $A = 0 \rightarrow 0 - 2 = -2 \rightarrow 24 \rightarrow Y$
 $B = 1 \rightarrow 1 - 2 = -1 \rightarrow 25 \rightarrow Z$
Result: VYZ

Shift 3

$X = 23 \rightarrow 23 - 3 = 20 \rightarrow U$
 $A = 0 \rightarrow 0 - 3 = -3 \rightarrow 23 \rightarrow X$
 $B = 1 \rightarrow 1 - 3 = -2 \rightarrow 24 \rightarrow Y$
Result: UXY

Shift 4

$X = 23 \rightarrow 23 - 4 = 19 \rightarrow T$
 $A = 0 \rightarrow 0 - 4 = -4 \rightarrow 22 \rightarrow W$
 $B = 1 \rightarrow 1 - 4 = -3 \rightarrow 23 \rightarrow X$
Result: TWX

Shift 5

$$X = 23 \rightarrow 23 - 5 = 18 \rightarrow S$$

$$A = 0 \rightarrow 0 - 5 = -5 \rightarrow 21 \rightarrow V$$

$$B = 1 \rightarrow 1 - 5 = -4 \rightarrow 22 \rightarrow W$$

Result: SVW

Shift 6

$$X = 23 \rightarrow 23 - 6 = 17 \rightarrow R$$

$$A = 0 \rightarrow 0 - 6 = -6 \rightarrow 20 \rightarrow U$$

$$B = 1 \rightarrow 1 - 6 = -5 \rightarrow 21 \rightarrow V$$

Result: RUV

Shift 7

$$X = 23 \rightarrow 23 - 7 = 16 \rightarrow Q$$

$$A = 0 \rightarrow 0 - 7 = -7 \rightarrow 19 \rightarrow T$$

$$B = 1 \rightarrow 1 - 7 = -6 \rightarrow 20 \rightarrow U$$

Result: QTU

Shift 8

$$X = 23 \rightarrow 23 - 8 = 15 \rightarrow P$$

$$A = 0 \rightarrow 0 - 8 = -8 \rightarrow 18 \rightarrow S$$

$$B = 1 \rightarrow 1 - 8 = -7 \rightarrow 19 \rightarrow T$$

Result: PST

Shift 9

$$X = 23 \rightarrow 23 - 9 = 14 \rightarrow O$$

$$A = 0 \rightarrow 0 - 9 = -9 \rightarrow 17 \rightarrow R$$

$$B = 1 \rightarrow 1 - 9 = -8 \rightarrow 18 \rightarrow S$$

Result: ORS

Shift 10

$$X = 23 \rightarrow 23 - 10 = 13 \rightarrow N$$

$$A = 0 \rightarrow 0 - 10 = -10 \rightarrow 16 \rightarrow Q$$

$$B = 1 \rightarrow 1 - 10 = -9 \rightarrow 17 \rightarrow R$$

Result: NQR

Shift 11

$$X = 23 \rightarrow 23 - 11 = 12 \rightarrow M$$

$$A = 0 \rightarrow 0 - 11 = -11 \rightarrow 15 \rightarrow P$$

$B = 1 \rightarrow 1 - 11 = -10 \rightarrow 16 \rightarrow Q$
Result: MPQ

Shift 12

$X = 23 \rightarrow 23 - 12 = 11 \rightarrow L$
 $A = 0 \rightarrow 0 - 12 = -12 \rightarrow 14 \rightarrow O$
 $B = 1 \rightarrow 1 - 12 = -11 \rightarrow 15 \rightarrow P$
Result: LOP

Shift 13

$X = 23 \rightarrow 23 - 13 = 10 \rightarrow K$
 $A = 0 \rightarrow 0 - 13 = -13 \rightarrow 13 \rightarrow N$
 $B = 1 \rightarrow 1 - 13 = -12 \rightarrow 14 \rightarrow O$
Result: KNO

Shift 14

$X = 23 \rightarrow 23 - 14 = 9 \rightarrow J$
 $A = 0 \rightarrow 0 - 14 = -14 \rightarrow 12 \rightarrow M$
 $B = 1 \rightarrow 1 - 14 = -13 \rightarrow 13 \rightarrow N$
Result: JMN

Shift 15

$X = 23 \rightarrow 23 - 15 = 8 \rightarrow I$
 $A = 0 \rightarrow 0 - 15 = -15 \rightarrow 11 \rightarrow L$
 $B = 1 \rightarrow 1 - 15 = -14 \rightarrow 12 \rightarrow M$
Result: ILM

Shift 16

$X = 23 \rightarrow 23 - 16 = 7 \rightarrow H$
 $A = 0 \rightarrow 0 - 16 = -16 \rightarrow 10 \rightarrow K$
 $B = 1 \rightarrow 1 - 16 = -15 \rightarrow 11 \rightarrow L$
Result: HKL

Shift 17

$X = 23 \rightarrow 23 - 17 = 6 \rightarrow G$
 $A = 0 \rightarrow 0 - 17 = -17 \rightarrow 9 \rightarrow J$
 $B = 1 \rightarrow 1 - 17 = -16 \rightarrow 10 \rightarrow K$
Result: GJK

Shift 18

$$X = 23 \rightarrow 23 - 18 = 5 \rightarrow F$$

$$A = 0 \rightarrow 0 - 18 = -18 \rightarrow 8 \rightarrow I$$

$$B = 1 \rightarrow 1 - 18 = -17 \rightarrow 9 \rightarrow J$$

Result: FIJ

Shift 19

$$X = 23 \rightarrow 23 - 19 = 4 \rightarrow E$$

$$A = 0 \rightarrow 0 - 19 = -19 \rightarrow 7 \rightarrow H$$

$$B = 1 \rightarrow 1 - 19 = -18 \rightarrow 8 \rightarrow I$$

Result: EHI

Shift 20

$$X = 23 \rightarrow 23 - 20 = 3 \rightarrow D$$

$$A = 0 \rightarrow 0 - 20 = -20 \rightarrow 6 \rightarrow G$$

$$B = 1 \rightarrow 1 - 20 = -19 \rightarrow 7 \rightarrow H$$

Result: DGH

Shift 21

$$X = 23 \rightarrow 23 - 21 = 2 \rightarrow C$$

$$A = 0 \rightarrow 0 - 21 = -21 \rightarrow 5 \rightarrow F$$

$$B = 1 \rightarrow 1 - 21 = -20 \rightarrow 6 \rightarrow G$$

Result: CFG

Shift 22

$$X = 23 \rightarrow 23 - 22 = 1 \rightarrow B$$

$$A = 0 \rightarrow 0 - 22 = -22 \rightarrow 4 \rightarrow E$$

$$B = 1 \rightarrow 1 - 22 = -21 \rightarrow 5 \rightarrow F$$

Result: BEF

Shift 23

$$X = 23 \rightarrow 23 - 23 = 0 \rightarrow A$$

$$A = 0 \rightarrow 0 - 23 = -23 \rightarrow 3 \rightarrow D$$

$$B = 1 \rightarrow 1 - 23 = -22 \rightarrow 4 \rightarrow E$$

Result: ADE

Shift 24

$$X = 23 \rightarrow 23 - 24 = -1 \rightarrow 25 \rightarrow Z$$

$$A = 0 \rightarrow 0 - 24 = -24 \rightarrow 2 \rightarrow C$$

$B = 1 \rightarrow 1 - 24 = -23 \rightarrow 3 \rightarrow D$

Result: ZCD

Shift 25

$X = 23 \rightarrow 23 - 25 = -2 \rightarrow 24 \rightarrow Y$

$A = 0 \rightarrow 0 - 25 = -25 \rightarrow 1 \rightarrow B$

$B = 1 \rightarrow 1 - 25 = -24 \rightarrow 2 \rightarrow C$

Result: YBC

The ciphertext XAB was decrypted by testing all 25 possible Caesar cipher shifts. None of the resulting plaintexts formed a recognizable English word, demonstrating that brute-force testing can efficiently explore the entire Caesar cipher keyspace, although the supplied ciphertext did not produce a meaningful plaintext

Exercise 3: Python function to encrypt and decrypt messages using the Caesar cipher.

Commands Used

```
mkdir -p ~/INT306_Cryptography
cd ~/INT306_Cryptography
pwd
nano caesar_cipher.py
```

Script:

```
def caesar_cipher(text, shift, mode="encrypt"):
    result = ""

    for char in text:
        if char.isalpha():
            start = ord('A') if char.isupper() else ord('a')
```

```
    if mode == "encrypt":
        shifted_char = chr((ord(char) - start + shift) % 26 + start)

    elif mode == "decrypt":
        shifted_char = chr((ord(char) - start - shift) % 26 + start)

    else:
        raise ValueError("Mode must be 'encrypt' or 'decrypt'")

    result += shifted_char

else:
    result += char

return result
```

```
plaintext = "CRYPTOGRAPHY"
shift = 7
```

```
encrypted_text = caesar_cipher(plaintext, shift, "encrypt")
print("Plaintext :", plaintext)
print("Encrypted :", encrypted_text)
```

```
decrypted_text = caesar_cipher(encrypted_text, shift, "decrypt")
print("Decrypted :", decrypted_text)
```

kali@olivil: ~/INT306_Cryptography

```
(kali@olivil)-[~/INT306_Cryptography]  
$ nano caesar_cipher.py
```

```
GNU nano 9.1 caesar_cipher.py *  
def caesar_cipher(text, shift, mode="encrypt"):  
    result = ""  
  
    for char in text:  
        if char.isalpha():  
            start = ord('A') if char.isupper() else ord('a')  
  
            if mode == "encrypt":  
                shifted_char = chr((ord(char) - start + shift) % 26 + start)  
  
            elif mode == "decrypt":  
                shifted_char = chr((ord(char) - start - shift) % 26 + start)  
  
            else:  
                raise ValueError("Mode must be 'encrypt' or 'decrypt'")  
  
            result += shifted_char  
  
        else:  
            result += char  
  
    return result  
  
plaintext = "CRYPTOGRAPHY"  
shift = 7  
  
encrypted_text = caesar_cipher(plaintext, shift, "encrypt")  
print("Plaintext :", plaintext)  
print("Encrypted :", encrypted_text)  
  
decrypted_text = caesar_cipher(encrypted_text, shift, "decrypt")  
print("Decrypted :", decrypted_text)
```

Command used:

python3 caesar_cipher.py

```
(kali@olivil)-[~/INT306_Cryptography]
$ python3 caesar_cipher.py
Plaintext : CRYPTOGRAPHY
Encrypted : JYFWAVNYHWOF
Decrypted : CRYPTOGRAPHY

(kali@olivil)-[~/INT306_Cryptography]
$
```

Implemented a Python-based Caesar cipher to encrypt and decrypt the plaintext CRYPTOGRAPHY using a shift of 7. The plaintext was encrypted to JYFWAVNYHWOF and successfully decrypted back to the original plaintext, demonstrating correct encryption and decryption.

2.2 The Vigenère Cipher

Exercises

1. Manual Decryption: Decrypt the ciphertext `LXFOPVEFRNHR` using the key `LEMON`.
2. Python Implementation: Write a Python script that takes a ciphertext and a key as input and returns the decrypted Vigenère plaintext.

Solution to Exercises

Exercises 1 Manual Decryption

Ciphertext: L X F O P V E F R N H R
Key: L E M O N L E M O N L E

Using: $P = (C - K) \bmod 26$

L(11) - L(11) = 0 → A
X(23) - E(4) = 19 → T
F(5) - M(12) = -7 → 19 → T
O(14) - O(14) = 0 → A
P(15) - N(13) = 2 → C
V(21) - L(11) = 10 → K
E(4) - E(4) = 0 → A
F(5) - M(12) = -7 → 19 → T
R(17) - O(14) = 3 → D
N(13) - N(13) = 0 → A
H(7) - L(11) = -4 → 22 → W
R(17) - E(4) = 13 → N

Result:

Ciphertext: LXFOPVEFRNHR

Key: LEMON

Plaintext: ATTACKATDAWN

The ciphertext LXFOPVEFRNHR was decrypted using the repeating key LEMON. Applying the formula $P = (C - K) \bmod 26$ produced the plaintext ATTACKATDAWN, confirming successful Vigenère decryption.

Exercises 2. Python Implementation

Commands Used:

nano vigenere_cipher.py

Script:

```
def vigenere_decrypt(ciphertext, key):  
    plaintext = ""  
    key = key.upper()  
    key_index = 0
```

```
for char in ciphertext:
    if char.isalpha():
        c = ord(char.upper()) - ord('A')
        k = ord(key[key_index % len(key)]) - ord('A')

        p = (c - k) % 26

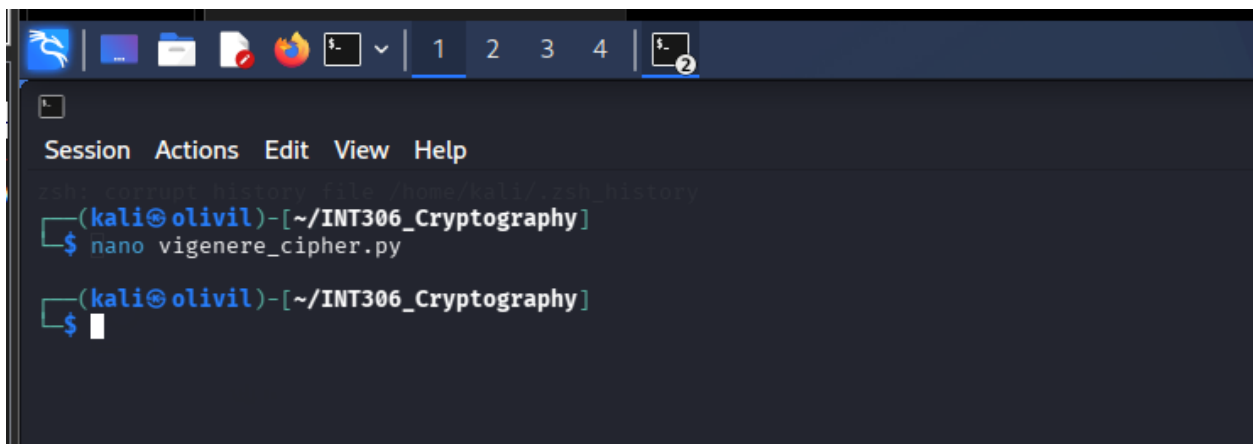
        plaintext += chr(p + ord('A'))
        key_index += 1
    else:
        plaintext += char

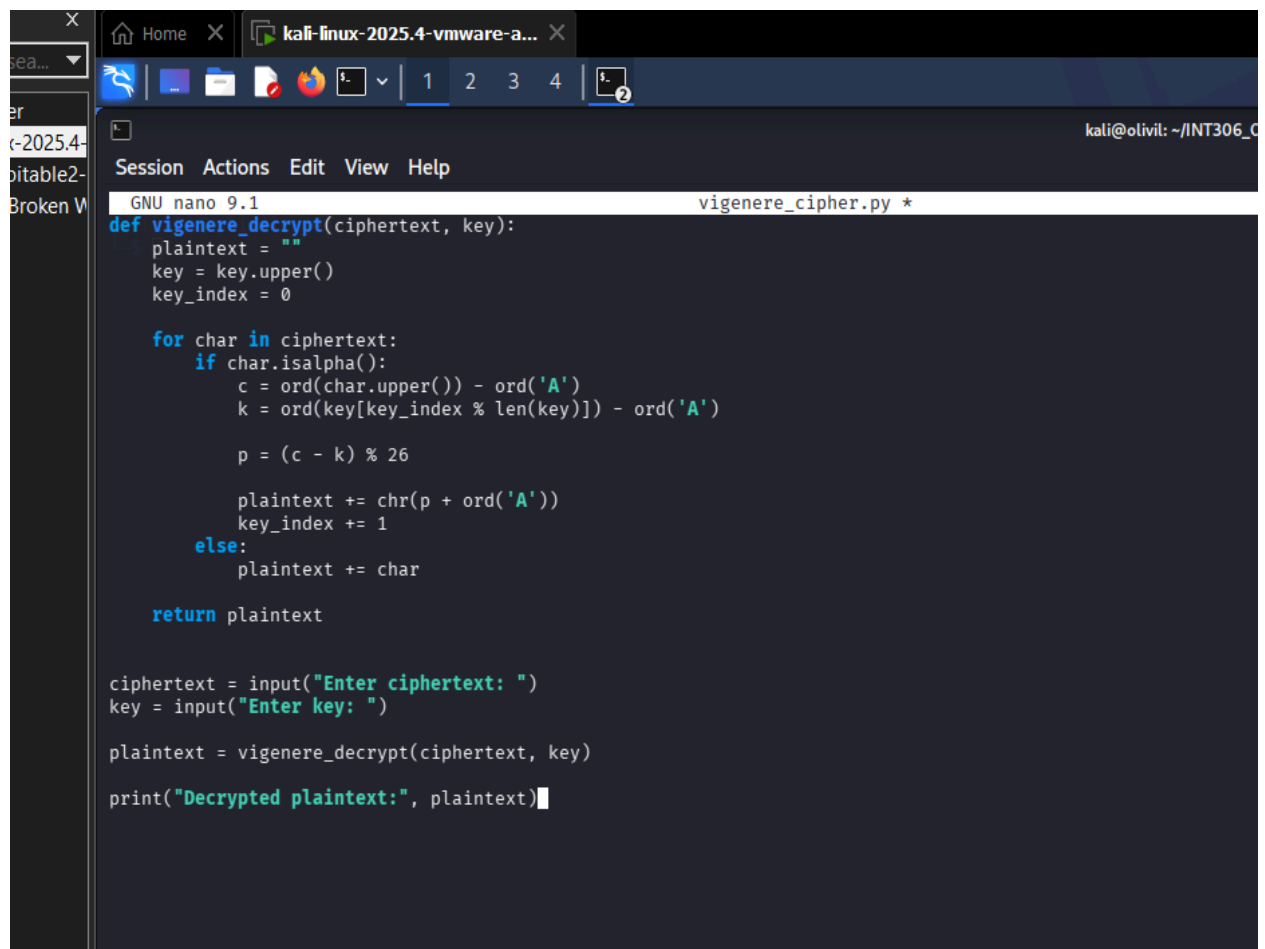
return plaintext
```

```
ciphertext = input("Enter ciphertext: ")
key = input("Enter key: ")

plaintext = vigenere_decrypt(ciphertext, key)

print("Decrypted plaintext:", plaintext)
```





The screenshot shows a terminal window with a nano text editor open. The editor is editing a file named `vigenere_cipher.py`. The code defines a `vigenere_decrypt` function that takes `ciphertext` and `key` as arguments. It converts the key to uppercase and iterates through each character of the ciphertext. For each alphabetic character, it calculates its position relative to 'A', subtracts the corresponding key character's position (also relative to 'A'), and wraps around using modulo 26 to find the plaintext character. Non-alphabetic characters are left unchanged. The script then prompts the user for ciphertext and key, calls the decryption function, and prints the result.

```
GNU nano 9.1 vigenere_cipher.py *
def vigenere_decrypt(ciphertext, key):
    plaintext = ""
    key = key.upper()
    key_index = 0

    for char in ciphertext:
        if char.isalpha():
            c = ord(char.upper()) - ord('A')
            k = ord(key[key_index % len(key)]) - ord('A')

            p = (c - k) % 26

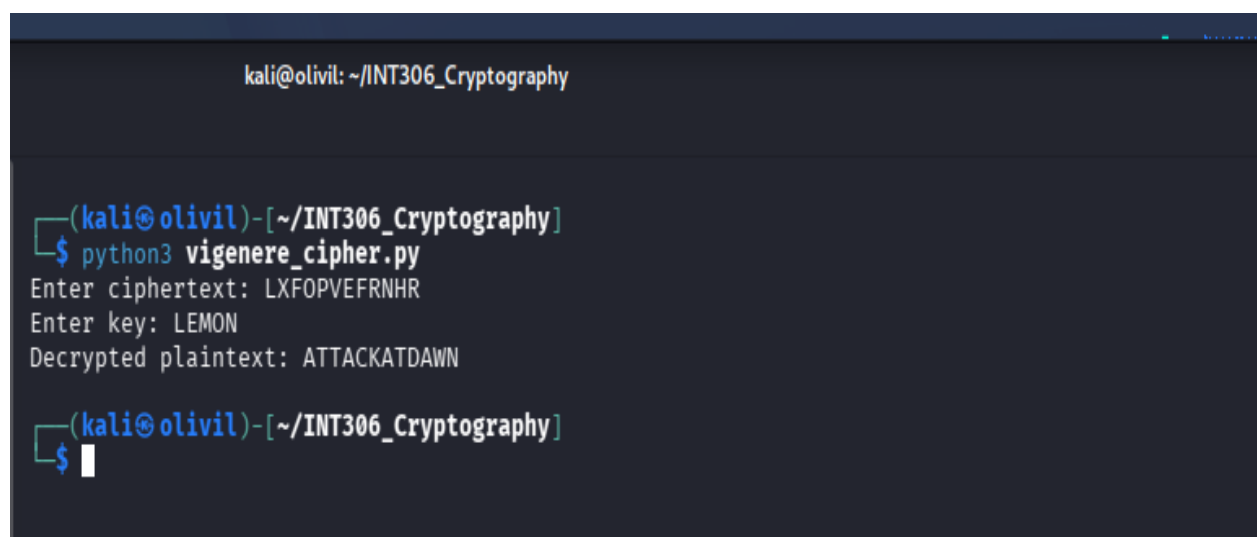
            plaintext += chr(p + ord('A'))
            key_index += 1
        else:
            plaintext += char

    return plaintext

ciphertext = input("Enter ciphertext: ")
key = input("Enter key: ")

plaintext = vigenere_decrypt(ciphertext, key)

print("Decrypted plaintext:", plaintext)
```



The screenshot shows a terminal window with the prompt `kali@olivil: ~/INT306_Cryptography`. The user runs `python3 vigenere_cipher.py`. The program prompts for ciphertext and key. The user enters `LXFOPVEFRNHR` for ciphertext and `LEMON` for the key. The program outputs `Decrypted plaintext: ATTACKATDAWN`. The prompt returns to `(kali@olivil)-[~/INT306_Cryptography]` with a new line ready for input.

```
kali@olivil: ~/INT306_Cryptography

(kali@olivil)-[~/INT306_Cryptography]
$ python3 vigenere_cipher.py
Enter ciphertext: LXFOPVEFRNHR
Enter key: LEMON
Decrypted plaintext: ATTACKATDAWN

(kali@olivil)-[~/INT306_Cryptography]
$
```

A Python Vigenère cipher decryption program was implemented to accept a ciphertext and key as user input. Using LXFOPVEFRNHR with the key LEMON, the program

successfully produced the plaintext ATTACKATDAWN.

3.1 Advanced Encryption Standard (AES)

Exercises

1. Use OpenSSL to encrypt a file using `AES-128-CBC`. What happens if you use the wrong password during decryption?
2. The ECB Vulnerability:** Download a `.bmp` image. Encrypt it using `AES-256-ECB` and `AES-256-CBC`. Compare the two encrypted images. Why can you still see the pattern in the ECB version?
3. Python AES-GCM:** Use the `cryptography` library to encrypt the string "Authenticated Data" using AES-GCM. Ensure you include an Initialization Vector (IV) and an Authentication Tag.
4. Secure File Encryption Tool (Making Things)

Problem:

Build a command-line tool in Python that can encrypt and decrypt files using AES-256-GCM. The tool should derive a key from a user-provided password.

Solution to Exercises

Exercise 1: OpenSSL to encrypt a file using `AES-128-CBC`

Commands Used:

```
echo "This is my secret message." > secret.txt
cat secret.txt
```

```
openssl enc -aes-128-cbc -salt -in secret.txt -out encrypted.bin -k mypassword
```

```
ls -l secret.txt encrypted.bin
```

```
openssl enc -d -aes-128-cbc -in encrypted.bin -out decrypted.txt -k mypassword  
cat decrypted.txt
```

```
openssl enc -d -aes-128-cbc -in encrypted.bin -out wrong.txt -k wrongpassword
```

```
ls -l
```

```
kali@olivil: ~/INT306_Cryptography

(kali@olivil)-[~/INT306_Cryptography]
$ echo "This is my secret message." > secret.txt

(kali@olivil)-[~/INT306_Cryptography]
$ cat secret.txt
This is my secret message.

(kali@olivil)-[~/INT306_Cryptography]
$ openssl enc -aes-128-cbc -salt -in secret.txt -out encrypted.bin -k mypassword
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(kali@olivil)-[~/INT306_Cryptography]
$ ls -l secret.txt encrypted.bin
-rw-rw-r-- 1 kali kali 48 Aug 17 23:15 encrypted.bin
-rw-rw-r-- 1 kali kali 27 Aug 17 23:10 secret.txt

(kali@olivil)-[~/INT306_Cryptography]
$ openssl enc -d -aes-128-cbc -in encrypted.bin -out decrypted.txt -k mypassword
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(kali@olivil)-[~/INT306_Cryptography]
$ cat decrypted.txt
This is my secret message.

(kali@olivil)-[~/INT306_Cryptography]
$ openssl enc -d -aes-128-cbc -in encrypted.bin -out wrong.txt -k wrongpassword
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
bad decrypt
80572663D67F0000:error:1C800064:Provider routines:ossl_cipher_unpadblock:bad decrypt:../providers/implementations/ciphers/ciphercommon_b
lock.c:107:

(kali@olivil)-[~/INT306_Cryptography]
$ ls -l
total 24
-rw-rw-r-- 1 kali kali 939 Aug 17 20:38 caesar_cipher.py
-rw-rw-r-- 1 kali kali 27 Aug 17 23:19 decrypted.txt
-rw-rw-r-- 1 kali kali 48 Aug 17 23:15 encrypted.bin
-rw-rw-r-- 1 kali kali 27 Aug 17 23:10 secret.txt
-rw-rw-r-- 1 kali kali 583 Aug 17 22:11 vigenere_cipher.py
-rw-rw-r-- 1 kali kali 16 Aug 17 23:19 wrong.txt

(kali@olivil)-[~/INT306_Cryptography]
$
```

- secret.txt contained the original plaintext: "This is my secret message."
- encrypted.bin was successfully created using AES-128-CBC.
- Decryption with the correct password successfully restored the original message in decrypted.txt.
- Decryption with the wrong password produced bad decrypt, showing that the incorrect password could not successfully decrypt the ciphertext.
- OpenSSL also displayed a warning recommending PBKDF2 instead of the deprecated key-derivation method.

Exercise 2: ECB Vulnerability

Commands used:

1. Created the BMP test image

```
magick -size 400x400 xc:white \
-fill black -draw "rectangle 50,50 350,350" \
-fill white -draw "rectangle 100,100 300,300" \
test.bmp
```

2. Checked the image

```
identify test.bmp
```

3. Encrypted using AES-256-ECB

```
openssl enc -aes-256-ecb -in test.bmp -out encrypted_ecb.bin -k
'MySecurePassword123'
```

4. Encrypted using AES-256-CBC

```
openssl enc -aes-256-cbc -salt -in test.bmp -out encrypted_cbc.bin -k  
'MySecurePassword123'
```

5. Checked the encrypted files

```
ls -lh test.bmp encrypted_ecb.bin encrypted_cbc.bin
```

6. Created the visual comparison script

```
nano ecb_cbc_demo.py
```

Script:

```
from pathlib import Path  
from Cryptodome.Cipher import AES  
  
# Read the BMP  
data = Path("test.bmp").read_bytes()  
  
# BMP header is 138 bytes for this image  
header = data[:138]  
  
# Actual pixel data  
pixels = data[138:]  
  
# AES-256 key  
key = b"MySecurePassword123".ljust(32, b"\0")  
  
# Pixel data is exactly divisible by 16,  
# so no padding is required.  
  
# AES-256-ECB  
ecb = AES.new(key, AES.MODE_ECB)  
ecb_pixels = ecb.encrypt(pixels)
```

```
# AES-256-CBC
iv = b"\0" * 16
cbc = AES.new(key, AES.MODE_CBC, iv)
cbc_pixels = cbc.encrypt(pixels)

# Rebuild BMP files using the original header
Path("ecb_visual.bmp").write_bytes(header + ecb_pixels)
Path("cbc_visual.bmp").write_bytes(header + cbc_pixels)

print("Correct ECB visual image created.")
print("Correct CBC visual image created.")
```

7. Ran the comparison script using:

```
python3 ecb_cbc_demo.py
```

8. Verified the visual BMP files

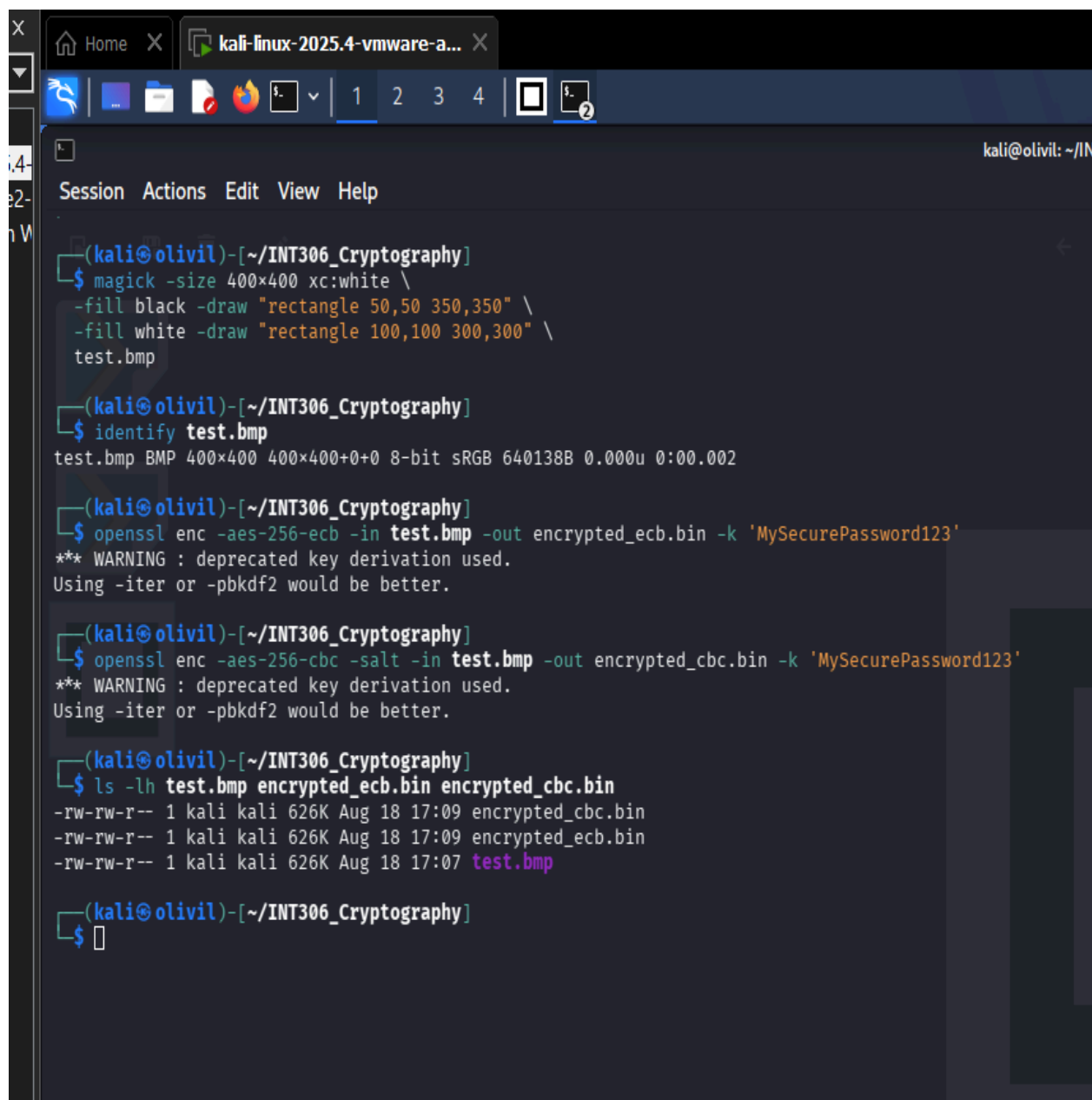
```
identify ecb_visual.bmp cbc_visual.bmp
```

9. Opened the ECB result

```
xdg-open ecb_visual.bmp
```

10. Opened the CBC result

```
xdg-open cbc_visual.bmp
```



The screenshot shows a Kali Linux terminal window with the title bar "kali-linux-2025.4-vmware-a...". The terminal is at the prompt "(kali@olivil)-[~/INT306_Cryptography]". The user runs the following commands:

```
(kali@olivil)-[~/INT306_Cryptography]
$ magick -size 400x400 xc:white \
  -fill black -draw "rectangle 50,50 350,350" \
  -fill white -draw "rectangle 100,100 300,300" \
  test.bmp

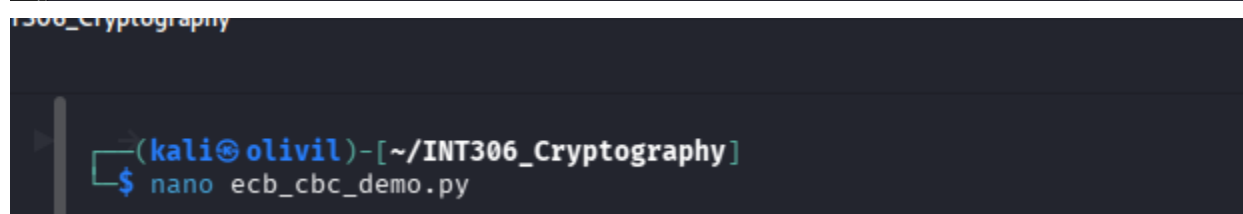
(kali@olivil)-[~/INT306_Cryptography]
$ identify test.bmp
test.bmp BMP 400x400 400x400+0+0 8-bit sRGB 640138B 0.000u 0:00.002

(kali@olivil)-[~/INT306_Cryptography]
$ openssl enc -aes-256-ecb -in test.bmp -out encrypted_ecb.bin -k 'MySecurePassword123'
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(kali@olivil)-[~/INT306_Cryptography]
$ openssl enc -aes-256-cbc -salt -in test.bmp -out encrypted_cbc.bin -k 'MySecurePassword123'
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(kali@olivil)-[~/INT306_Cryptography]
$ ls -lh test.bmp encrypted_ecb.bin encrypted_cbc.bin
-rw-rw-r-- 1 kali kali 626K Aug 18 17:09 encrypted_cbc.bin
-rw-rw-r-- 1 kali kali 626K Aug 18 17:09 encrypted_ecb.bin
-rw-rw-r-- 1 kali kali 626K Aug 18 17:07 test.bmp

(kali@olivil)-[~/INT306_Cryptography]
$
```



The screenshot shows the continuation of the terminal window. The user has entered the command to open a file in nano:

```
(kali@olivil)-[~/INT306_Cryptography]
$ nano ecb_cbc_demo.py
```

```
GNU nano 9.1                                ecb_cbc_demo.py *
from pathlib import Path
from Cryptodome.Cipher import AES

# Read the BMP
data = Path("test.bmp").read_bytes()

# BMP header is 138 bytes for this image
header = data[:138]

# Actual pixel data
pixels = data[138:]

# AES-256 key
key = b"MySecurePassword123".ljust(32, b"\0")

# Pixel data is exactly divisible by 16,
# so no padding is required.

# AES-256-ECB
ecb = AES.new(key, AES.MODE_ECB)
ecb_pixels = ecb.encrypt(pixels)

# AES-256-CBC
iv = b"\0" * 16
cbc = AES.new(key, AES.MODE_CBC, iv)
cbc_pixels = cbc.encrypt(pixels)

# Rebuild BMP files using the original header
Path("ecb_visual.bmp").write_bytes(header + ecb_pixels)
Path("cbc_visual.bmp").write_bytes(header + cbc_pixels)

print("Correct ECB visual image created.")
print("Correct CBC visual image created.")
```

```
(kali@olivil)-[~/INT306_Cryptography]
```

```
$ python3 ecb_cbc_demo.py
```

```
Correct ECB visual image created.
```

```
Correct CBC visual image created.
```

```
(kali@olivil)-[~/INT306_Cryptography]
```

```
$ identify ecb_visual.bmp cbc_visual.bmp
```

```
ecb_visual.bmp BMP 400x400 400x400+0+0 8-bit sRGB 640138B 0.000u 0:00.002
```

```
cbc_visual.bmp BMP 400x400 400x400+0+0 8-bit sRGB 640138B 0.000u 0:00.002
```

```
(kali@olivil)-[~/INT306_Cryptography]
```

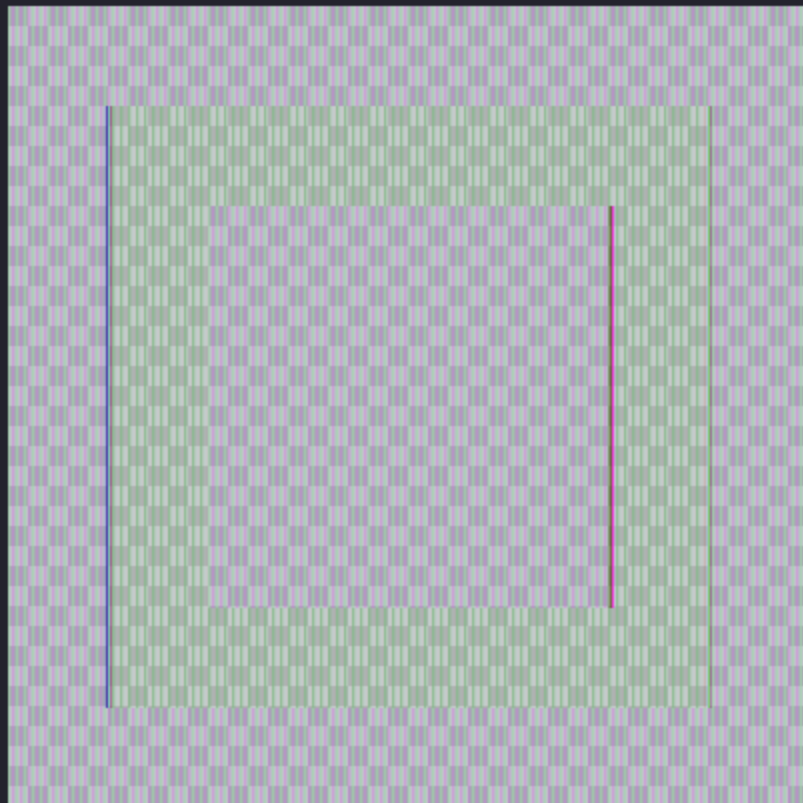
```
$ xdg-open ecb_visual.bmp
```

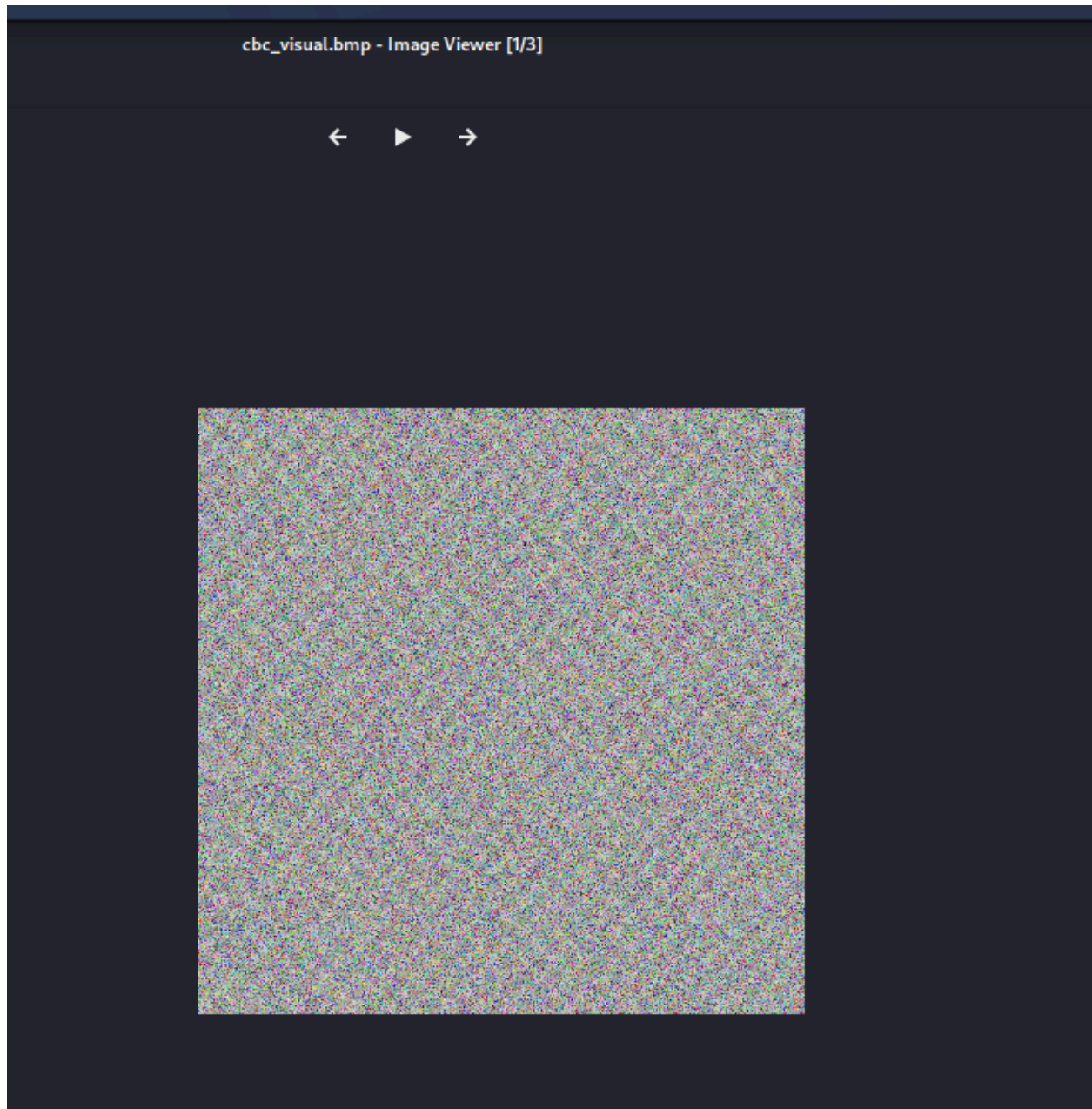
```
(kali@olivil)-[~/INT306_Cryptography]
```

```
$ xdg-open cbc_visual.bmp
```

```
(kali@olivil)-[~/INT306_Cryptography]
```

```
$ █
```





The AES-256-ECB encrypted image retained the original square pattern, while the AES-256-CBC encrypted image appeared noisy and concealed the original pattern. This occurs because ECB encrypts identical plaintext blocks independently, producing identical ciphertext blocks and therefore leaking patterns. CBC chains blocks together, preventing the same plaintext blocks from producing the same ciphertext pattern.

Exercise 3: Python AES-GCM

Commands used:

1. Checked that AES-GCM is available

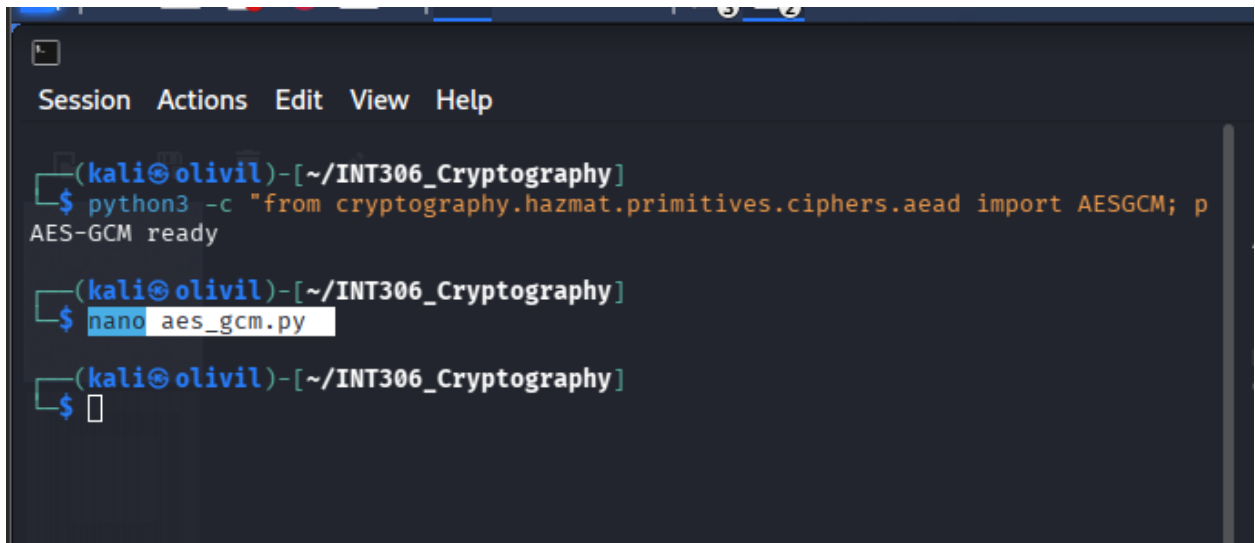
```
python3 -c "from cryptography.hazmat.primitives.ciphers.aead import AESGCM; print('AES-GCM ready')"
```

2. Created the Python AES-GCM script

```
nano aes_gcm.py
```

3. Ran the AES-GCM encryption program

```
python3 aes_gcm.py
```



```
Session Actions Edit View Help

(kali@olivil)-[~/INT306_Cryptography]
$ python3 -c "from cryptography.hazmat.primitives.ciphers.aead import AESGCM; p
AES-GCM ready

(kali@olivil)-[~/INT306_Cryptography]
$ nano aes_gcm.py

(kali@olivil)-[~/INT306_Cryptography]
$
```

```
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

# Generate a random 256-bit AES key
key = AESGCM.generate_key(bit_length=256)

# Generate a random 12-byte IV/nonce
iv = os.urandom(12)

# Create AES-GCM object
aesgcm = AESGCM(key)

# Message
plaintext = b"Authenticated Data"

# Encrypt
encrypted = aesgcm.encrypt(iv, plaintext, None)

# AES-GCM appends a 16-byte authentication tag
ciphertext = encrypted[:-16]
auth_tag = encrypted[-16:]

print("AES-256-GCM Encryption")
print("-----")
print("Plaintext      :", plaintext.decode())
print("Key            :", key.hex())
print("IV/Nonce       :", iv.hex())
print("Ciphertext     :", ciphertext.hex())
print("Authentication Tag:", auth_tag.hex())
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Loca
^/ Go T

```
kali@olivil: ~/INT306_Cryptography

(kali@olivil)-[~/INT306_Cryptography]
$ python3 aes_gcm.py
AES-256-GCM Encryption

Plaintext : Authenticated Data
Key       : 43dfaa7548fbd45a33dba360058aac7f90f8a191c71c2126b1214dd4a752bf44
Nonce/IV  : 698e8ab909488f34e3a68b3a
Ciphertext: edde239c9924750674f2524ab8c7b8e845923ae3ed712b925e9e0e4e6dd406717dd1

(kali@olivil)-[~/INT306_Cryptography]
$
```

AES-256-GCM was used to encrypt the message Authenticated Data. The encryption generated a 256-bit key and a 12-byte random IV/nonce, producing ciphertext that also contains the GCM authentication tag. AES-GCM provides both confidentiality and integrity, allowing tampering with the encrypted data to be detected.

Exercise 4: Secure File Encryption Tool

Commands used

1. Checked PBKDF2 and AES-GCM

```
python3 -c "from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC; from cryptography.hazmat.primitives.ciphers.aead import AESGCM; print('PBKDF2 and AES-GCM ready')"
```

2. Created the plaintext file

```
echo "This is my secret file for the INT306 AES-256-GCM lab." >  
my_secret_file.txt
```

3. Viewed the plaintext

```
cat my_secret_file.txt
```

4. Created the Python encryption tool

```
nano secure_file.py
```

script:

```
import os  
from cryptography.hazmat.primitives.ciphers.aead import AESGCM  
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC  
from cryptography.hazmat.primitives import hashes
```

```
def derive_key(password, salt):  
    kdf = PBKDF2HMAC(  
        algorithm=hashes.SHA256(),  
        length=32,  
        salt=salt,  
        iterations=100000  
    )  
    return kdf.derive(password.encode())
```

```
def encrypt_file(file_path, password):  
    salt = os.urandom(16)
```

```

nonce = os.urandom(12)

key = derive_key(password, salt)
aesgcm = AESGCM(key)

with open(file_path, "rb") as f:
    plaintext = f.read()

ciphertext = aesgcm.encrypt(nonce, plaintext, None)

output_file = file_path + ".enc"

with open(output_file, "wb") as f:
    f.write(salt + nonce + ciphertext)

print(f"Encrypted: {output_file}")
print("Salt :", salt.hex())
print("Nonce:", nonce.hex())

def decrypt_file(file_path, password):
    with open(file_path, "rb") as f:
        data = f.read()

    salt = data[:16]
    nonce = data[16:28]
    ciphertext = data[28:]

    key = derive_key(password, salt)
    aesgcm = AESGCM(key)

    try:
        plaintext = aesgcm.decrypt(nonce, ciphertext, None)

        output_file = file_path.replace(".enc", ".dec")

        with open(output_file, "wb") as f:

```

```
f.write(plaintext)

print(f"Decrypted: {output_file}")

except Exception:
    print("Decryption failed: incorrect password or modified file.")

# Encrypt the file
encrypt_file("my_secret_file.txt", "MySecurePassword123")

# Decrypt using the correct password
decrypt_file("my_secret_file.txt.enc", "MySecurePassword123")

# Test with the wrong password
decrypt_file("my_secret_file.txt.enc", "WrongPassword123")
```

5. Run the encryption/decryption tool

```
python3 secure_file.py
```

6. Checked the generated files

```
ls -lh my_secret_file.txt my_secret_file.txt.enc my_secret_file.txt.dec
```

7. Displayed the decrypted file

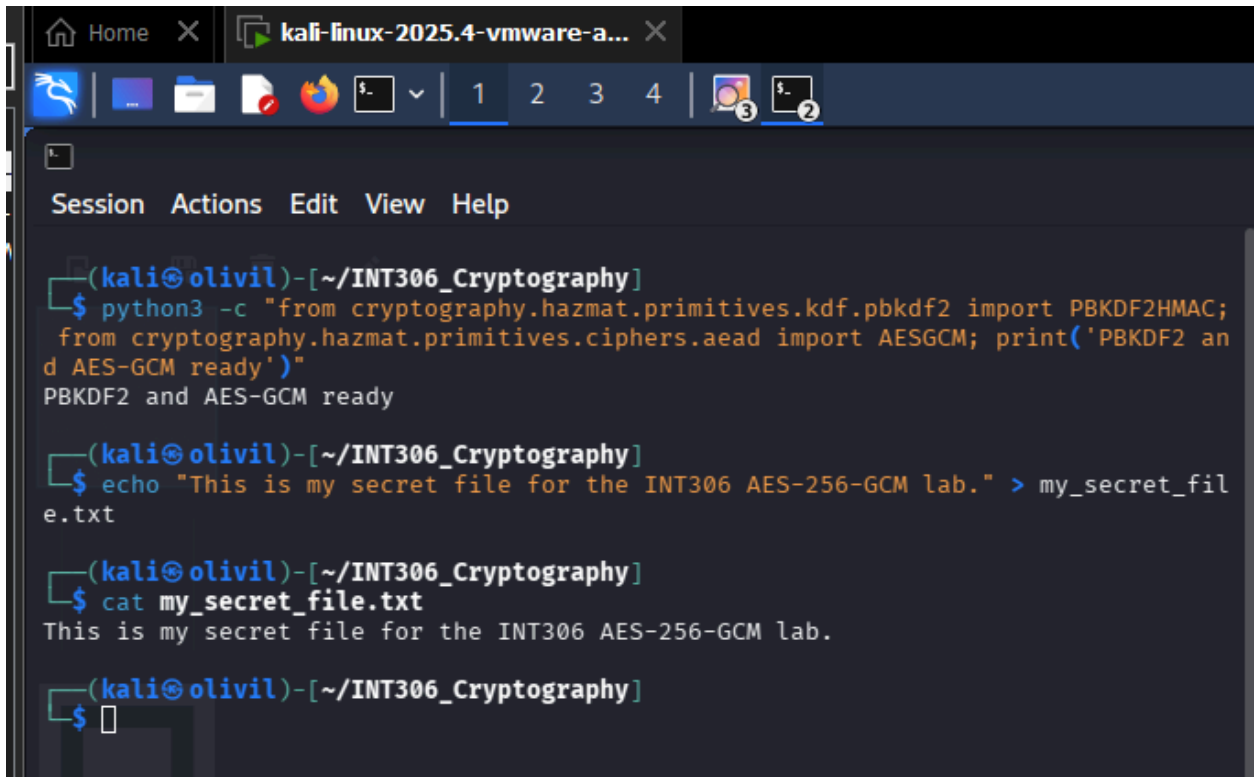
```
cat my_secret_file.txt.dec
```

8. Compared the original and decrypted files

```
cmp my_secret_file.txt my_secret_file.txt.dec
```

9. Verified using SHA-256

```
sha256sum my_secret_file.txt my_secret_file.txt.dec
```



The screenshot shows a Kali Linux terminal window with the title bar 'kali-linux-2025.4-vmware-a...'. The terminal prompt is '(kali@olivil)-[~/INT306_Cryptography]'. The user runs a Python command to import PBKDF2HMAC and AESGCM from the cryptography library, which outputs 'PBKDF2 and AES-GCM ready'. Then, the user runs 'echo "This is my secret file for the INT306 AES-256-GCM lab." > my_secret_file.txt'. Finally, the user runs 'cat my_secret_file.txt', which outputs 'This is my secret file for the INT306 AES-256-GCM lab.'.

```
(kali@olivil)-[~/INT306_Cryptography]
$ python3 -c "from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC;
from cryptography.hazmat.primitives.ciphers.aead import AESGCM; print('PBKDF2 and
AES-GCM ready')"
PBKDF2 and AES-GCM ready

(kali@olivil)-[~/INT306_Cryptography]
$ echo "This is my secret file for the INT306 AES-256-GCM lab." > my_secret_file.txt

(kali@olivil)-[~/INT306_Cryptography]
$ cat my_secret_file.txt
This is my secret file for the INT306 AES-256-GCM lab.

(kali@olivil)-[~/INT306_Cryptography]
$
```

kali@olivil: ~/INT306_Cryptography

GNU nano 9.1

secure_file.py *

```
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes
```

```
def derive_key(password, salt):
    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=32,
        salt=salt,
        iterations=100000
    )
    return kdf.derive(password.encode())

def encrypt_file(file_path, password):
    salt = os.urandom(16)
    nonce = os.urandom(12)

    key = derive_key(password, salt)
    aesgcm = AESGCM(key)

    with open(file_path, "rb") as f:
        plaintext = f.read()

    ciphertext = aesgcm.encrypt(nonce, plaintext, None)

    output_file = file_path + ".enc"

    with open(output_file, "wb") as f:
        f.write(salt + nonce + ciphertext)

    print(f"Encrypted: {output_file}")
    print("Salt :", salt.hex())
    print("Nonce:", nonce.hex())

def decrypt_file(file_path, password):
    with open(file_path, "rb") as f:
        data = f.read()

    salt = data[:16]
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U
M-E

```
kali@olivil: ~/INT306_Cryptography

(kali@olivil)-[~/INT306_Cryptography]
$ nano secure_file.py

(kali@olivil)-[~/INT306_Cryptography]
$ python3 secure_file.py
Encrypted: my_secret_file.txt.enc
Salt : 4d8690720b9b185ae6e79415b4dc95f1
Nonce: 10eaadc50efacabe06786aee
Decrypted: my_secret_file.txt.dec
Decryption failed: incorrect password or modified file.

(kali@olivil)-[~/INT306_Cryptography]
$ ls -lh my_secret_file.txt my_secret_file.txt.enc my_secret_file.txt.dec
-rw-rw-r-- 1 kali kali 55 Aug 18 19:40 my_secret_file.txt
-rw-rw-r-- 1 kali kali 55 Aug 18 19:46 my_secret_file.txt.dec
-rw-rw-r-- 1 kali kali 99 Aug 18 19:46 my_secret_file.txt.enc

(kali@olivil)-[~/INT306_Cryptography]
$ cat my_secret_file.txt.dec
This is my secret file for the INT306 AES-256-GCM lab.

(kali@olivil)-[~/INT306_Cryptography]
$ cmp my_secret_file.txt my_secret_file.txt.dec

(kali@olivil)-[~/INT306_Cryptography]
$ sha256sum my_secret_file.txt my_secret_file.txt.dec
e46a6d73de14190f7fe76ad5ef12787df0389c16257c01de07a69e271696022d my_secret_file.txt
e46a6d73de14190f7fe76ad5ef12787df0389c16257c01de07a69e271696022d my_secret_file.txt.dec

(kali@olivil)-[~/INT306_Cryptography]
$
```

A Python-based AES-256-GCM file encryption tool was implemented using PBKDF2 for password-based key derivation. The file was successfully encrypted and decrypted with the correct password, while decryption with an incorrect password failed. The matching SHA-256 hashes confirmed that the decrypted file was identical to the original.

4.1 Cryptographic Hash Functions

Exercises

1. Using Python or OpenSSL, compute the hash values for the following strings:
 - "Hello World"
 - "Secure Hashing"
 - "INT306 Cryptography"

Record the MD5 and SHA-256 hash values for each. Discuss the differences in the output length and characteristics.

2. Download a file (e.g., a Linux ISO) and find its official SHA-256 checksum. Verify the integrity of your downloaded file using `sha256sum`.

3. Create a file named `original.txt` with the content: `The quick brown fox jumps over the lazy dog.`

- Compute its SHA-256 hash.
- Create a new file named `modified.txt` with a single character changed: `The quick brown fox jumps over the lazy cog.`
- Compute the SHA-256 hash of `modified.txt`.
- Compare the two hashes. Discuss how drastically the hash changes due to a minor input alteration, illustrating the **avalanche effect**.

4. Use OpenSSL to generate an HMAC-SHA256 for the message "Financial Transaction: \$500" using the secret key "super-secret-key-123".

5. Write a Python script that:

- Takes a password as input.
- Hashes the password using `bcrypt` with a randomly generated salt.
- Stores the hash.
- Later, takes another password input and verifies it against the stored hash.

Discuss why `bcrypt` is preferred over simple SHA-256 for password hashing.

6. Research and describe how a rainbow table works to crack password hashes. Explain why salting passwords effectively mitigates rainbow table attacks.

7. Demonstrate an MD5 hash collision using known collision pairs. Explain why this makes MD5 unsuitable for integrity checks where collision resistance is critical.

Task: Obtain two files that are known to produce an MD5 collision (e.g., from a reputable source like the [MD5 Collision

Project](<https://www.win.tue.nl/~bdeweger/Collisions/>)). Verify their hashes and explain the implications for digital signatures

- Task:

Implement the Python script to generate a rainbow table for at least 5 common passwords. Then, given an MD5 hash (e.g.,

`d8578edf8458ce06fbc5bb76a587711`), use your table to find the original`

password. Discuss the limitations of this simple rainbow table and how salting would prevent this attack.

- Task: Select an application (e.g., Git, Digital Forensics, Secure Boot, Certificate Transparency) and prepare a one-page report (approx. 300-500 words) explaining the role of hash functions, including specific algorithms used, their benefits, and any associated vulnerabilities or real-world implications.

Exercise 1: Compute Hash Values

Commands used:

1. The Python hash script

```
nano hash_values.py
```

script:

```
import hashlib
```

```
messages = [  
    "Hello World",  
    "Secure Hashing",  
    "INT306 Cryptography"  
]
```

```
for message in messages:
```

```
    data = message.encode("utf-8")
```

```
    md5_hash = hashlib.md5(data).hexdigest()
```

```
    sha256_hash = hashlib.sha256(data).hexdigest()
```

```
    print("Message:", message)
```

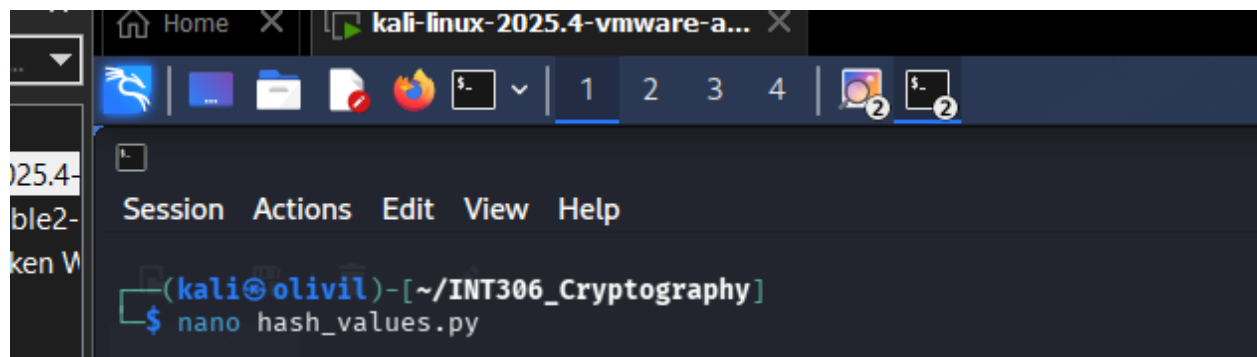
```
    print("MD5   :", md5_hash)
```

```
    print("SHA-256:", sha256_hash)
```

```
    print("-" * 50)
```

2. Script used:

python3 hash_values.py



```
GNU nano 9.1 hash_values.py
import hashlib

messages = [
    "Hello World",
    "Secure Hashing",
    "INT306 Cryptography"
]

for message in messages:
    data = message.encode("utf-8")

    md5_hash = hashlib.md5(data).hexdigest()
    sha256_hash = hashlib.sha256(data).hexdigest()

    print("Message:", message)
    print("MD5      :", md5_hash)
    print("SHA-256:", sha256_hash)
    print("-" * 50)
```

```
(kali@olivil) - [~/INT306_Cryptography]
$ python3 hash_values.py
Message: Hello World
MD5      : b10a8db164e0754105b7a99be72e3fe5
SHA-256: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
-----
Message: Secure Hashing
MD5      : 55a5bc5ee6b414cfcca1c67ba6f76eed
SHA-256: 4efbe2da50e67c855b3894c099e590a8480e7a35c6cd66f729cbe6bbfddf0a36
-----
Message: INT306 Cryptography
MD5      : 1d5a7039d3c83aed53aa21215e795fe8
SHA-256: dd743616105be6f4e7d4d6aedf2d9f999fe0cf0a8a60fcf9f98d3c92ec3ba572
-----
(kali@olivil) - [~/INT306_Cryptography]
$
```

MD5 and SHA-256 hashes were successfully generated for the three specified messages. MD5 produces a 128-bit digest represented by 32 hexadecimal characters,

while SHA-256 produces a 256-bit digest represented by 64 hexadecimal characters. SHA-256 provides stronger collision resistance and is preferred over MD5 for modern security applications

Exercise 2: File Integrity Verification

Commands used

1. Create the test file

```
echo "INT306 File Integrity Test" > integrity_test.txt
```

2. Generate the SHA-256 checksum

```
sha256sum integrity_test.txt
```

3. Saved the checksum

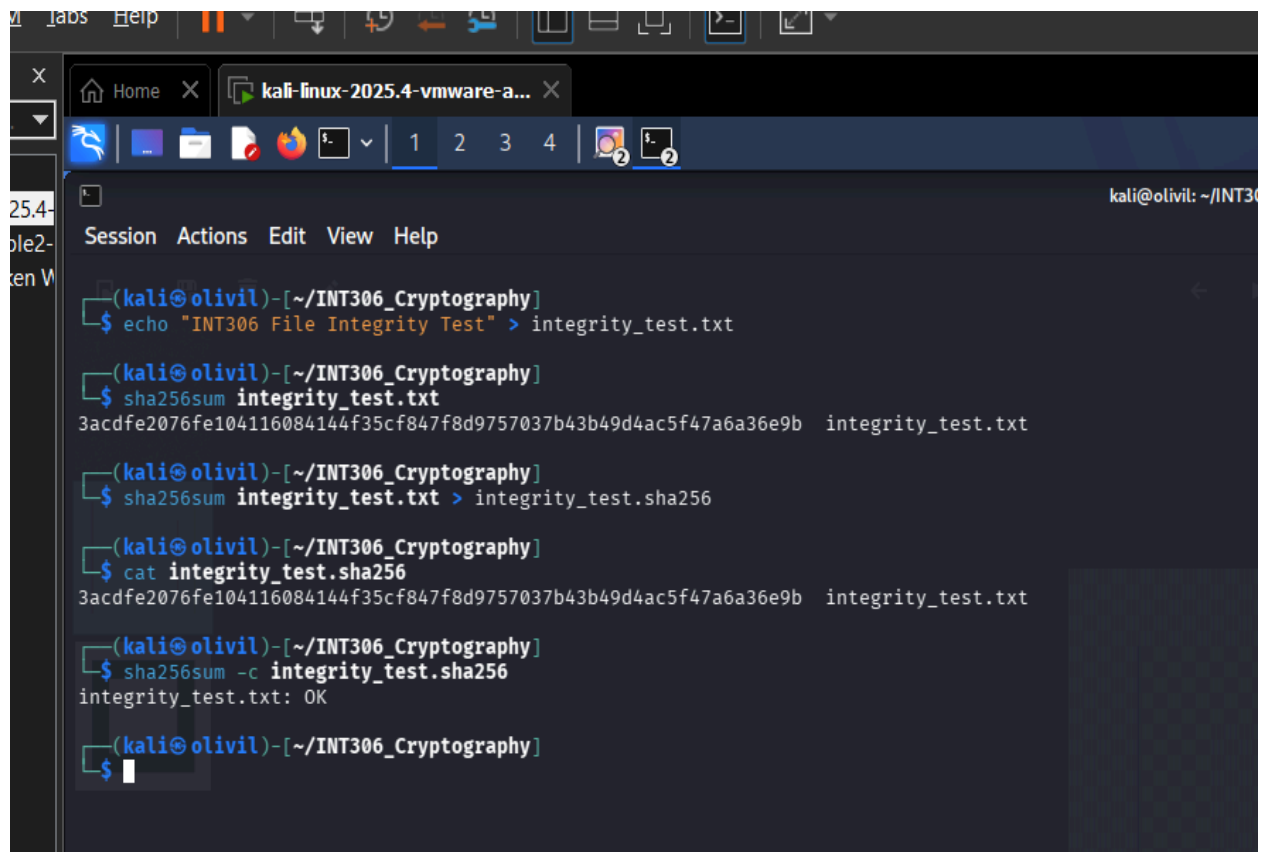
```
sha256sum integrity_test.txt > integrity_test.sha256
```

4. Viewed the saved checksum

```
cat integrity_test.sha256
```

5. Verified the file integrity

```
sha256sum -c integrity_test.sha256
```



```
(kali@olivil)-[~/INT306_Cryptography]
$ echo "INT306 File Integrity Test" > integrity_test.txt

(kali@olivil)-[~/INT306_Cryptography]
$ sha256sum integrity_test.txt
3acdfe2076fe104116084144f35cf847f8d9757037b43b49d4ac5f47a6a36e9b integrity_test.txt

(kali@olivil)-[~/INT306_Cryptography]
$ sha256sum integrity_test.txt > integrity_test.sha256

(kali@olivil)-[~/INT306_Cryptography]
$ cat integrity_test.sha256
3acdfe2076fe104116084144f35cf847f8d9757037b43b49d4ac5f47a6a36e9b integrity_test.txt

(kali@olivil)-[~/INT306_Cryptography]
$ sha256sum -c integrity_test.sha256
integrity_test.txt: OK

(kali@olivil)-[~/INT306_Cryptography]
$
```

The SHA-256 checksum of integrity_test.txt was generated and stored in a checksum file. The file was then verified using sha256sum -c, which returned OK, confirming that the file contents matched the original checksum and had not been altered.

Exercise 3: Hash Function Properties (Avalanche Effect)

Commands used:

1. Create the original file.

```
echo "The quick brown fox jumps over the lazy dog." > original.txt
```

2. Calculates its SHA-256 hash.

```
sha256sum original.txt
```

3. Creates a second file with one character changed.

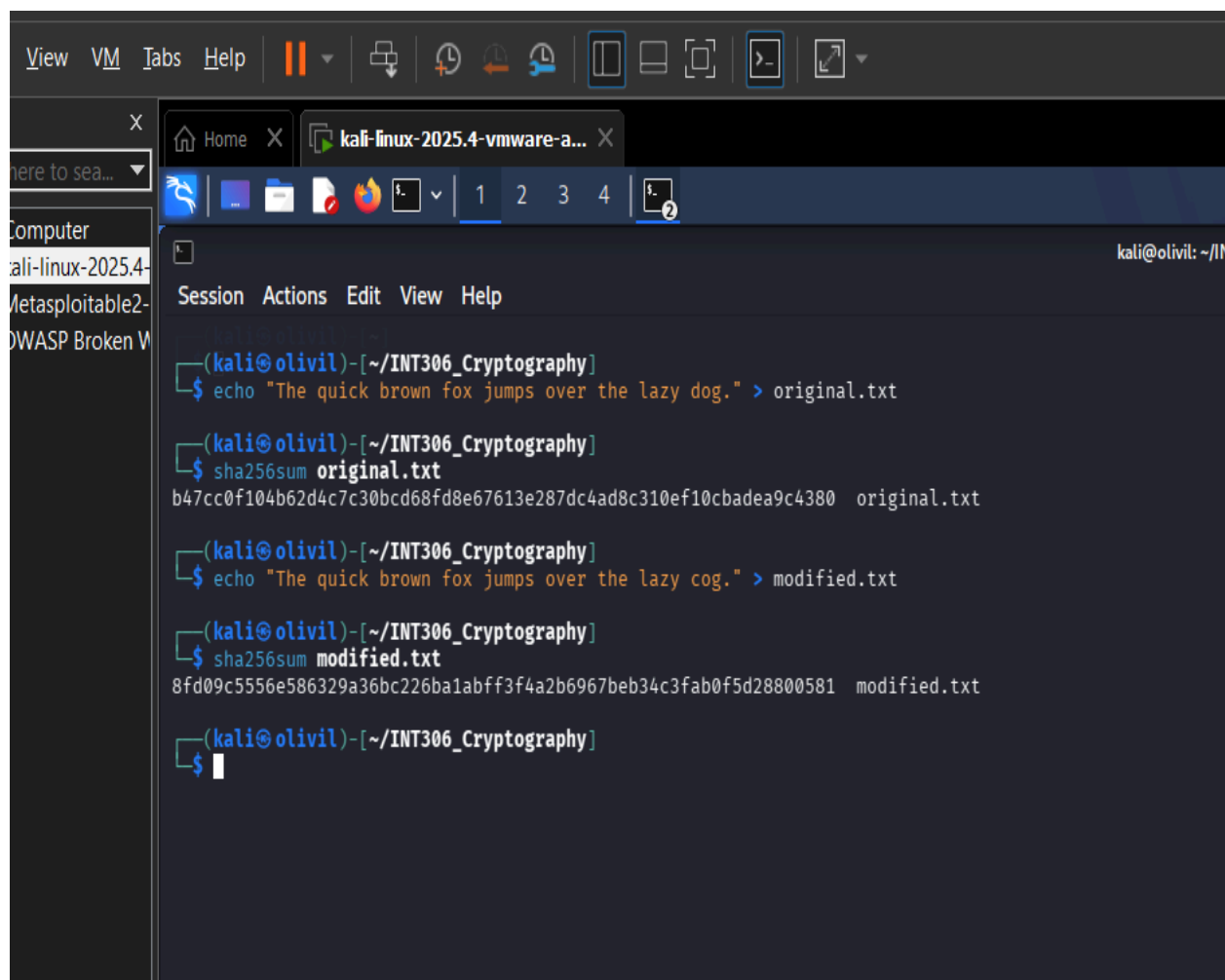
changing only one character:

I changed from dog → cog

```
echo "The quick brown fox jumps over the lazy cog." > modified.txt
```

4. Calculates the SHA-256 hash of the modified file.

```
sha256sum modified.txt
```

A screenshot of a Kali Linux terminal window. The window title is 'kali-linux-2025.4-vmware-a...'. The terminal shows a series of commands and their outputs. First, the user creates a file 'original.txt' with the text 'The quick brown fox jumps over the lazy dog.'. Then, they calculate the SHA-256 hash of 'original.txt', which is 'b47cc0f104b62d4c7c30bcd68fd8e67613e287dc4ad8c310ef10cbadea9c4380'. Next, they create a file 'modified.txt' with the text 'The quick brown fox jumps over the lazy cog.', changing the last character from 'dog' to 'cog'. Finally, they calculate the SHA-256 hash of 'modified.txt', which is '8fd09c5556e586329a36bc226ba1abff3f4a2b6967beb34c3fab0f5d28800581'. The terminal output clearly shows that a single character change results in a completely different hash, demonstrating the avalanche effect.

```
(kali@olivil)-[~/INT306_Cryptography]
$ echo "The quick brown fox jumps over the lazy dog." > original.txt

(kali@olivil)-[~/INT306_Cryptography]
$ sha256sum original.txt
b47cc0f104b62d4c7c30bcd68fd8e67613e287dc4ad8c310ef10cbadea9c4380  original.txt

(kali@olivil)-[~/INT306_Cryptography]
$ echo "The quick brown fox jumps over the lazy cog." > modified.txt

(kali@olivil)-[~/INT306_Cryptography]
$ sha256sum modified.txt
8fd09c5556e586329a36bc226ba1abff3f4a2b6967beb34c3fab0f5d28800581  modified.txt

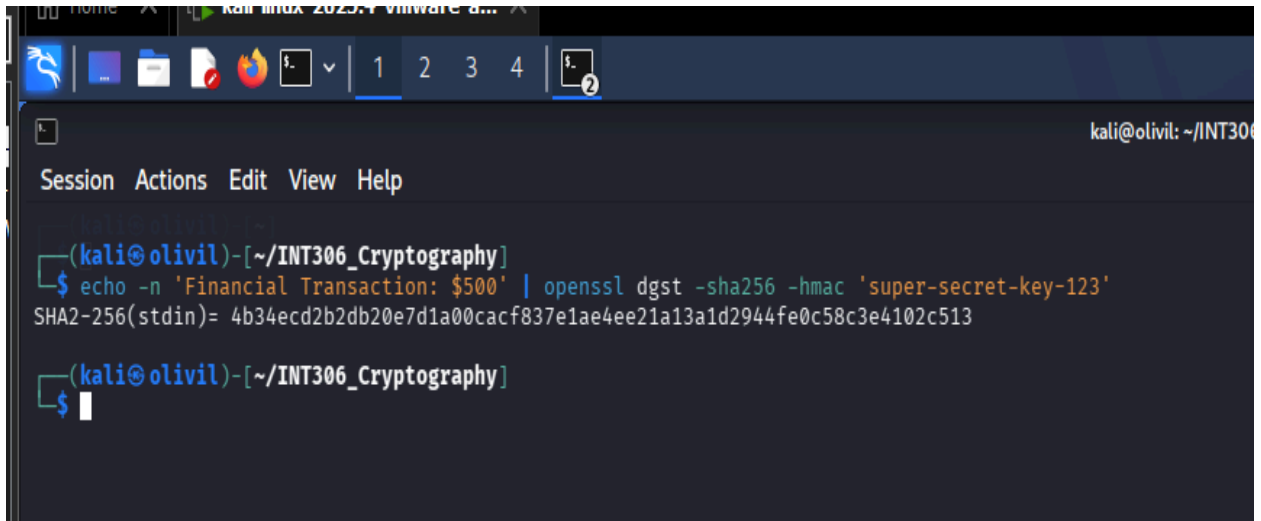
(kali@olivil)-[~/INT306_Cryptography]
$
```

The avalanche effect was demonstrated by changing only one character in the original message, from **dog** to **cog**. Despite this very small change, the SHA-256 hash changed completely from **b47cc0f...** to **8fd09c55...**, demonstrating that small changes to input produce significantly different hash outputs.

Exercise 4: HMAC Generation

Command used:

```
echo -n 'Financial Transaction: $500' | openssl dgst -sha256 -hmac  
'super-secret-key-123'
```



The screenshot shows a terminal window with a dark background. At the top, there is a window title bar with icons for a terminal, a file manager, and a web browser. Below the title bar, there is a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal prompt is '(kali@olivil)-[~/INT306_Cryptography]'. The command being executed is 'echo -n 'Financial Transaction: \$500' | openssl dgst -sha256 -hmac 'super-secret-key-123''. The output of the command is 'SHA2-256(stdin)= 4b34ecd2b2db20e7d1a00cacf837e1ae4ee21a13a1d2944fe0c58c3e4102c513'. The prompt is then shown again with a cursor.

```
(kali@olivil)-[~/INT306_Cryptography]  
$ echo -n 'Financial Transaction: $500' | openssl dgst -sha256 -hmac 'super-secret-key-123'  
SHA2-256(stdin)= 4b34ecd2b2db20e7d1a00cacf837e1ae4ee21a13a1d2944fe0c58c3e4102c513  
  
(kali@olivil)-[~/INT306_Cryptography]  
$
```

An HMAC-SHA256 was generated for the message "Financial Transaction: \$500" using the secret key super-secret-key-123. The resulting HMAC was **4b34ecd2b2db20e7d1a00cacf837e1ae4ee21a13a1d2944fe0c58c3e4102c513**, demonstrating how a secret key can be combined with a hash function to provide message authentication and integrity.

Exercise 5: Password Security with bcrypt

Commands Used:

1 Create the bcrypt demonstration script

```
nano bcrypt_demo.py
```

Bash:

```

import bcrypt

password = b"MySecurePassword123"

# Generate two bcrypt hashes using the same password
hash1 = bcrypt.hashpw(password, bcrypt.gensalt())
hash2 = bcrypt.hashpw(password, bcrypt.gensalt())

# Store the first bcrypt hash
with open("stored_hash.txt", "wb") as file:
    file.write(hash1)

print("Password:", password.decode())
print("Hash 1:", hash1.decode())
print("Hash 2:", hash2.decode())

# Verify the correct password
print("Correct password:", bcrypt.checkpw(password, hash1))

# Verify an incorrect password
wrong_password = b"WrongPassword123"
print("Wrong password:", bcrypt.checkpw(wrong_password, hash1))

# Read the stored hash
with open("stored_hash.txt", "rb") as file:
    stored_hash = file.read()

# Take another password input and verify it against the stored hash
check_password = input("Enter password to verify: ")

if bcrypt.checkpw(check_password.encode(), stored_hash):
    print("Password is correct!")
else:
    print("Password is incorrect!")

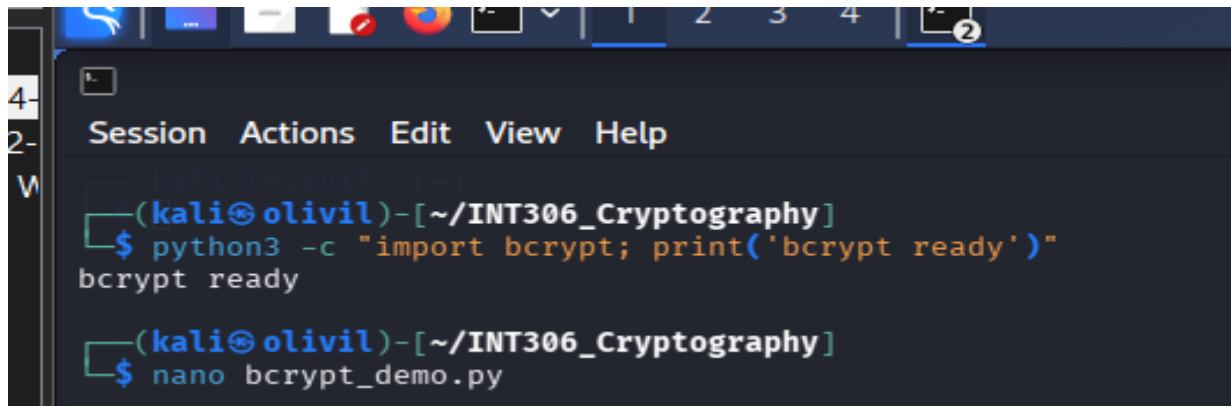
```

Saved:

Ctrl + O → Enter → Ctrl + X

Then:

python3 bcrypt_demo.py

A screenshot of a Kali Linux terminal window. The window has a dark background with a menu bar at the top containing 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows the user 'olivil' at the 'kali' prompt in the directory '~/INT306_Cryptography'. The first command executed is 'python3 -c "import bcrypt; print(\'bcrypt ready\')"', which outputs 'bcrypt ready'. The second command is 'nano bcrypt_demo.py'.

```
(kali@olivil)-[~/INT306_Cryptography]
$ python3 -c "import bcrypt; print('bcrypt ready')"
bcrypt ready

(kali@olivil)-[~/INT306_Cryptography]
$ nano bcrypt_demo.py
```

6_Cryptography

```
(kali@olivil)-[~/INT306_Cryptography]
$ python3 bcrypt_demo.py
Password: MySecurePassword123
Hash 1: $2b$12$W0uJfd/iQ.LdbJ0Xxp5p0l/b6WuwzSHHKhepTIPVnZuaEw98n0Gi
Hash 2: $2b$12$h6lu2/m2AO.djfG.yn0FJ.LZ7RyDxTj3zIrgQkL3Q/j6wZX60Glu0
Correct password: True
Wrong password: False
Enter password to verify: MySecurePassword123
Password is correct!

(kali@olivil)-[~/INT306_Cryptography]
$ python3 bcrypt_demo.py
Password: MySecurePassword123
Hash 1: $2b$12$ExdLqWu.sSXVPf0iUHemxu50SdSEJ60hux3AfUzLH8.I3260GcUoa
Hash 2: $2b$12$81DgCX.Z4bpI3VYpn4RF7uapjWgxZa/j5c.m5fWDNLJSpnqIblqVa
Correct password: True
Wrong password: False
Enter password to verify: WrongPassword123
Password is incorrect!

(kali@olivil)-[~/INT306_Cryptography]
$ █
```

The results show that bcrypt successfully stored and verified the password. The correct password was accepted, while the wrong password was rejected.

Why bcrypt is preferred: bcrypt is designed for password hashing and is intentionally slower, making brute-force attacks more difficult than with fast algorithms like SHA-256. It also uses a unique salt, so the same password produces different hashes.

```
GNU nano 9.1                                bcrypt_demo.py *
import bcrypt

password = b"MySecurePassword123"

# Generate two bcrypt hashes using the same password
hash1 = bcrypt.hashpw(password, bcrypt.gensalt())
hash2 = bcrypt.hashpw(password, bcrypt.gensalt())

# Store the first bcrypt hash
with open("stored_hash.txt", "wb") as file:
    file.write(hash1)

print("Password:", password.decode())
print("Hash 1:", hash1.decode())
print("Hash 2:", hash2.decode())

# Verify the correct password
print("Correct password:", bcrypt.checkpw(password, hash1))

# Verify an incorrect password
wrong_password = b"WrongPassword123"
print("Wrong password:", bcrypt.checkpw(wrong_password, hash1))

# Read the stored hash
with open("stored_hash.txt", "rb") as file:
    stored_hash = file.read()

# Take another password input and verify it against the stored hash
check_password = input("Enter password to verify: ")

if bcrypt.checkpw(check_password.encode(), stored_hash):
    print("Password is correct!")
else:
    print("Password is incorrect!")
```

```
GNU nano 9.1                                bcrypt_demo.py *
import bcrypt

password = b"MySecurePassword123"

# Generate two bcrypt hashes using the same password
hash1 = bcrypt.hashpw(password, bcrypt.gensalt())
hash2 = bcrypt.hashpw(password, bcrypt.gensalt())

# Store the first bcrypt hash
with open("stored_hash.txt", "wb") as file:
    file.write(hash1)

print("Password:", password.decode())
print("Hash 1:", hash1.decode())
print("Hash 2:", hash2.decode())

# Verify the correct password
print("Correct password:", bcrypt.checkpw(password, hash1))

# Verify an incorrect password
wrong_password = b"WrongPassword123"
print("Wrong password:", bcrypt.checkpw(wrong_password, hash1))

# Read the stored hash
with open("stored_hash.txt", "rb") as file:
    stored_hash = file.read()

# Take another password input and verify it against the stored hash
check_password = input("Enter password to verify: ")

if bcrypt.checkpw(check_password.encode(), stored_hash):
    print("Password is correct!")
else:
    print("Password is incorrect!")
```

The output shows that the password was successfully hashed using bcrypt. When the correct password was entered, it was accepted as **True**, while the wrong password was rejected as **False**, showing that bcrypt is working properly for password verification.

6. Rainbow tables

A rainbow table is a precomputed database containing large numbers of password hashes and their corresponding possible passwords. An attacker can compare a stolen password hash against the table instead of calculating hashes from scratch for every password guess.

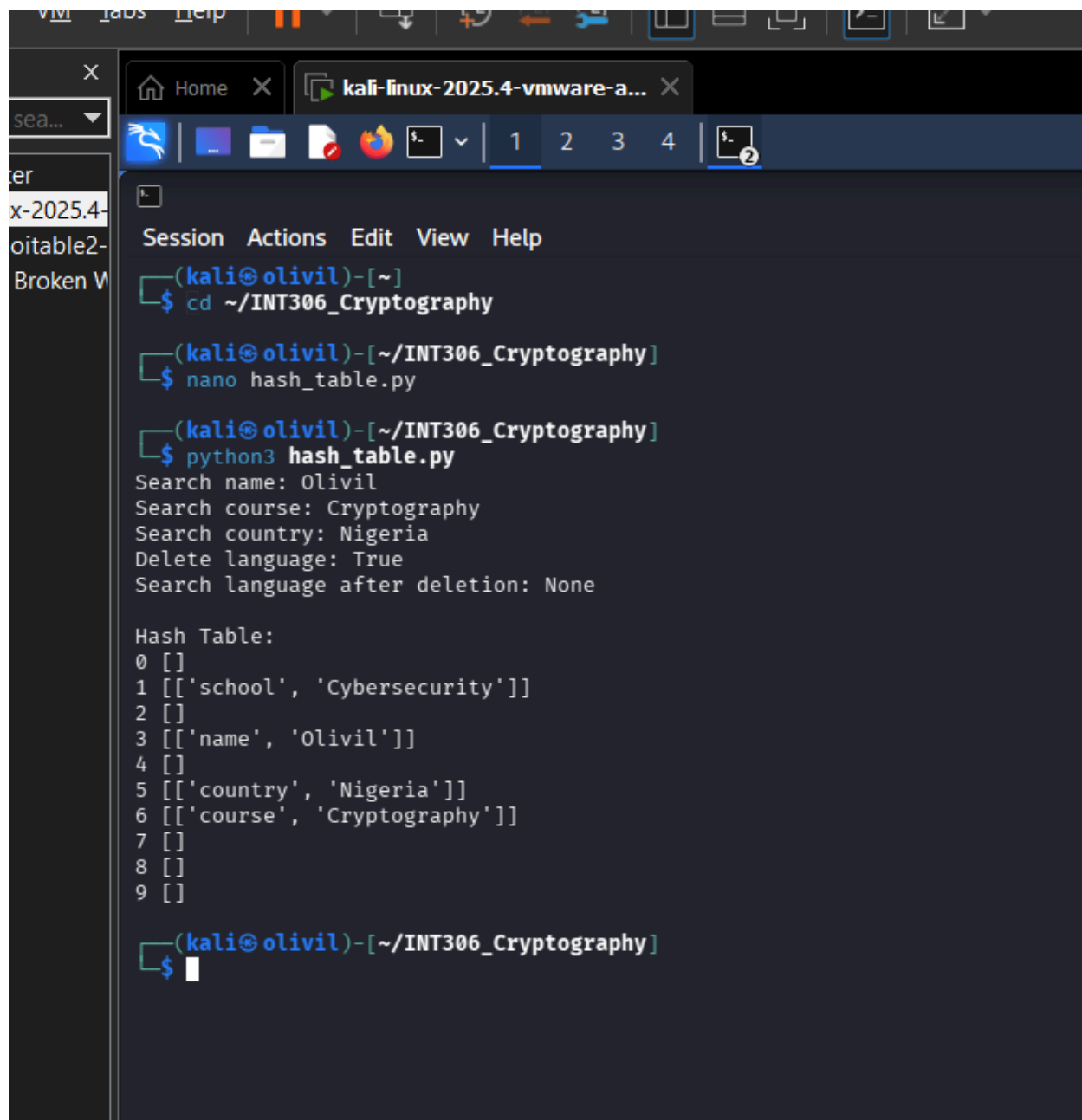
Why salting helps: a salt is a random value added to a password before hashing. Because each password gets a different salt, the same password produces different hashes, making precomputed rainbow tables ineffective against properly salted password hashes.

For the practical part, we can demonstrate this with a small controlled example rather than using an actual password-cracking database.

7. MD5 collision

An MD5 collision occurs when two different inputs produce the same MD5 hash. This is possible because MD5's collision resistance has been broken.

This makes MD5 unsuitable for integrity checks where collision resistance is important: an attacker could potentially create two different files with the same MD5 value, meaning the hash alone cannot reliably prove that a file has not been maliciously substituted.



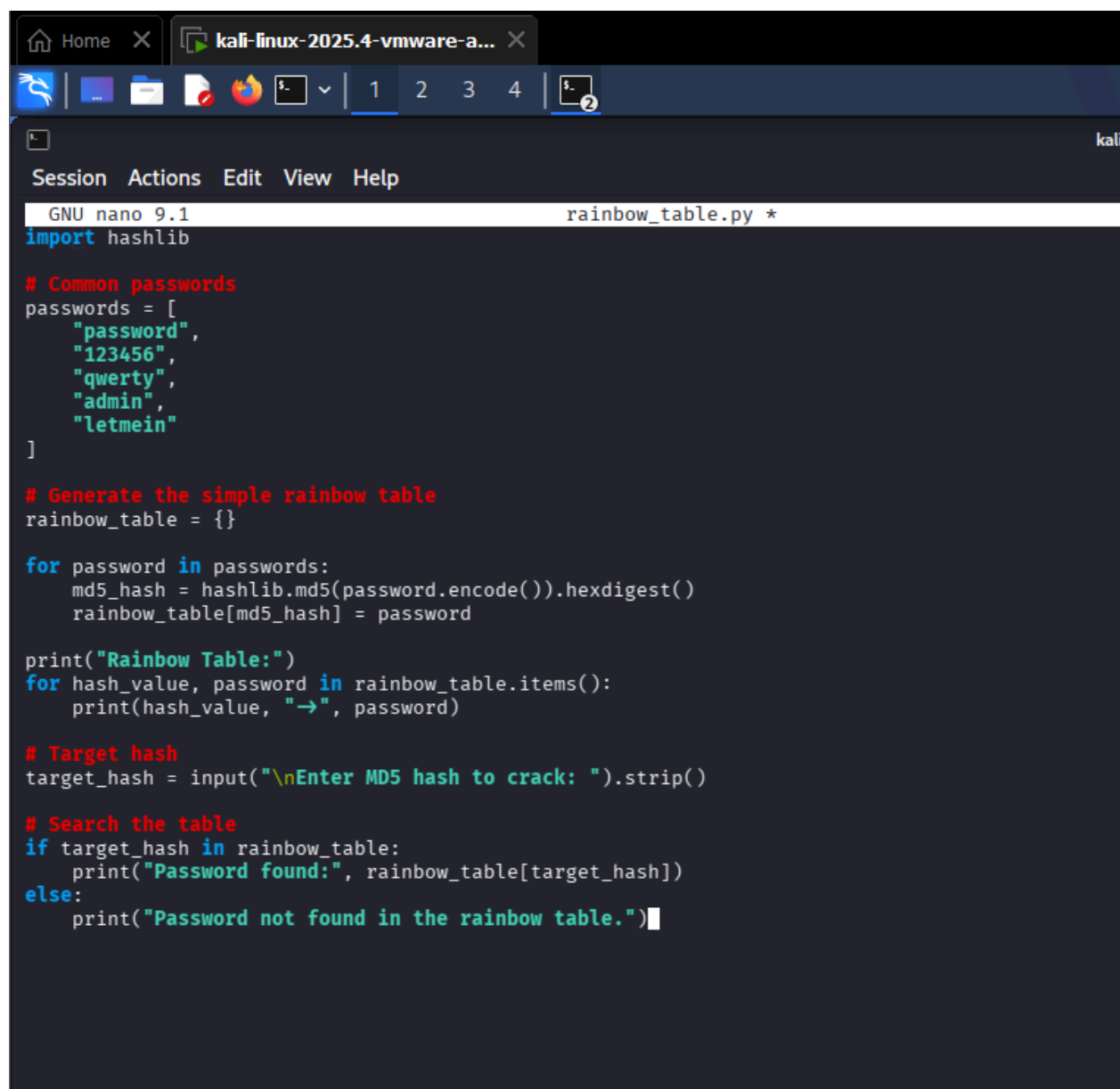
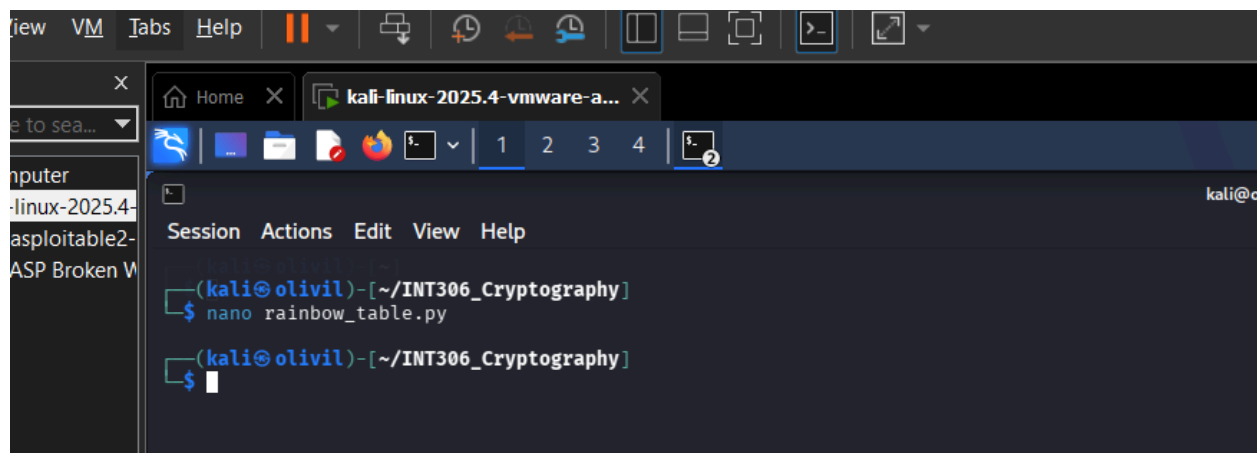
```
(kali@olivil)-[~]
$ cd ~/INT306_Cryptography

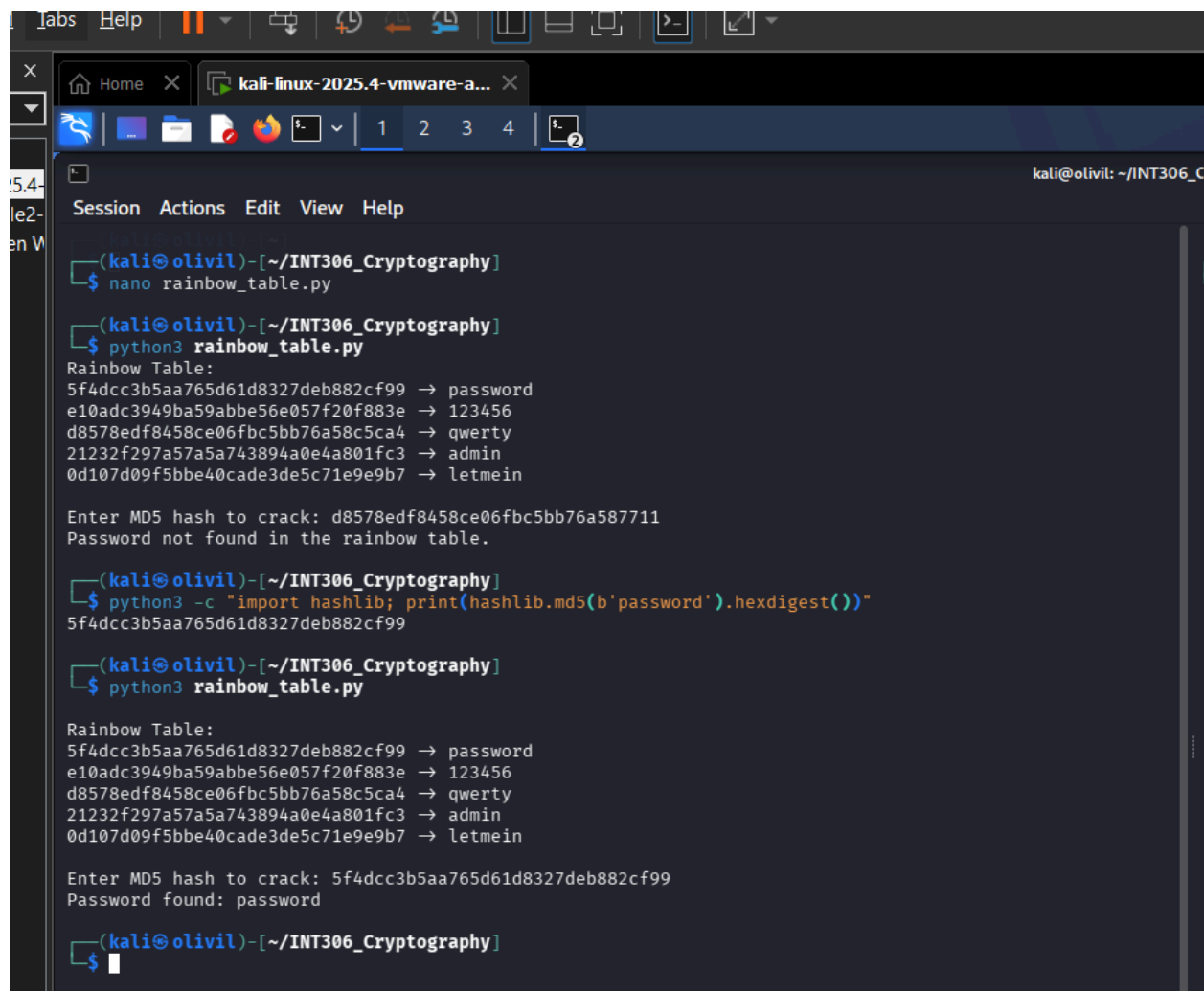
(kali@olivil)-[~/INT306_Cryptography]
$ nano hash_table.py

(kali@olivil)-[~/INT306_Cryptography]
$ python3 hash_table.py
Search name: Olivil
Search course: Cryptography
Search country: Nigeria
Delete language: True
Search language after deletion: None

Hash Table:
0 []
1 [['school', 'Cybersecurity']]
2 []
3 [['name', 'Olivil']]
4 []
5 [['country', 'Nigeria']]
6 [['course', 'Cryptography']]
7 []
8 []
9 []

(kali@olivil)-[~/INT306_Cryptography]
$
```





```
(kali@olivil)-[~/INT306_Cryptography]
$ nano rainbow_table.py

(kali@olivil)-[~/INT306_Cryptography]
$ python3 rainbow_table.py
Rainbow Table:
5f4dcc3b5aa765d61d8327deb882cf99 → password
e10adc3949ba59abbe56e057f20f883e → 123456
d8578edf8458ce06fbc5bb76a58c5ca4 → qwerty
21232f297a57a5a743894a0e4a801fc3 → admin
0d107d09f5bbe40cade3de5c71e9e9b7 → letmein

Enter MD5 hash to crack: d8578edf8458ce06fbc5bb76a587711
Password not found in the rainbow table.

(kali@olivil)-[~/INT306_Cryptography]
$ python3 -c "import hashlib; print(hashlib.md5(b'password').hexdigest())"
5f4dcc3b5aa765d61d8327deb882cf99

(kali@olivil)-[~/INT306_Cryptography]
$ python3 rainbow_table.py

Rainbow Table:
5f4dcc3b5aa765d61d8327deb882cf99 → password
e10adc3949ba59abbe56e057f20f883e → 123456
d8578edf8458ce06fbc5bb76a58c5ca4 → qwerty
21232f297a57a5a743894a0e4a801fc3 → admin
0d107d09f5bbe40cade3de5c71e9e9b7 → letmein

Enter MD5 hash to crack: 5f4dcc3b5aa765d61d8327deb882cf99
Password found: password

(kali@olivil)-[~/INT306_Cryptography]
$
```

A simple rainbow table was created using MD5 hashes of five common passwords. When a hash corresponding to one of the stored passwords was provided, the script successfully recovered the original password by matching the hash in the table. This demonstrates how rainbow tables can crack unsalted password hashes, while salting would make precomputed tables ineffective.

Cryptographic Hash Functions in Version Control: The Case of Git

1. Introduction and Role of Hash Functions

In modern software development, Version Control Systems (VCS) like Git rely heavily on cryptographic hash functions to ensure data integrity, facilitate efficient data tracking, and prevent undetected tampering. In Git, hash functions act as unique content identifiers. Rather than referencing files, commits, or directory trees by arbitrary filenames or version numbers, Git uses a content-addressable storage mechanism. Every object stored in Git—whether a file blob, a tree (directory structure), a commit, or an annotated tag—is assigned a unique cryptographic hash derived directly from its content.

2. Specific Algorithms Used

- **SHA-1 (Secure Hash Algorithm 1):** Historically, Git used SHA-1, which produces a 160-bit (20-byte) hash value represented as a 40-character hexadecimal string. For example, every commit hash long used by developers consists of a unique SHA-1 digest.
 - **SHA-256 (Transitioning Phase):** Due to security vulnerabilities inherent to SHA-1, modern Git implementations have added support for SHA-256, producing a 256-bit digest for improved cryptographic security.
-

3. Key Benefits

- **Data Integrity and Immutability:** Because every commit hash is computed using the commit's content, author metadata, timestamp, and the hash of its parent commit, altering even a single character in history changes all subsequent hashes. This creates an append-only DAG (Directed Acyclic Graph) of commits.

- **Fast Deduplication:** Git can instantly determine if two files or snapshots are identical by comparing their fixed-size hash values without reading through full file contents.
 - **Tamper Evident Architecture:** Cryptographic hashing makes unauthorized modifications to project history immediately detectable across distributed repositories.
-

4. Vulnerabilities and Real-World Implications

While cryptographic hashes provide strong guarantees, older algorithms present severe security risks:

- **Collision Attacks (SHAttered Attack):** In 2017, researchers from CWI Amsterdam and Google successfully executed the first real-world collision attack against SHA-1 (known as the *SHAttered* attack). They produced two distinct PDF documents that generated the exact same SHA-1 hash value.
- **Implications for Git:** A successful hash collision allows a malicious actor to craft a benign code commit and a malicious commit that yield identical SHA-1 hashes. An attacker could potentially replace legitimate source code with malicious payload without altering the repository's commit graph, bypassing basic integrity checks.
- **Mitigations:** In response, Git developers introduced collision detection mechanisms (like `sha1dc`) to detect and reject active collision attempts. Furthermore, Git introduced experimental and gradual support for SHA-256 object formats to future-proof repository security against expanding computational capabilities.

Bash:

9.HashTable class in Python

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    # Hash function
    def _hash(self, key):
        return hash(key) % self.size

    # Insert a key-value pair
    def insert(self, key, value):
        index = self._hash(key)

        # Check if the key already exists
        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return

        # Add new key-value pair
        self.table[index].append([key, value])

    # Search for a key
    def search(self, key):
        index = self._hash(key)

        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]

        return None
```

```
# Delete a key-value pair
def delete(self, key):
    index = self._hash(key)

    for pair in self.table[index]:
        if pair[0] == key:
            self.table[index].remove(pair)
            return True

    return False
```

```
# Create a hash table
ht = HashTable()
```

```
# Insert 5 different key-value pairs
ht.insert("name", "Olivil")
ht.insert("course", "Cryptography")
ht.insert("school", "Cybersecurity")
ht.insert("country", "Nigeria")
ht.insert("language", "Python")
```

```
# Test search operation
print("Search name:", ht.search("name"))
print("Search course:", ht.search("course"))
print("Search country:", ht.search("country"))
```

```
# Test delete operation
print("Delete language:", ht.delete("language"))
```

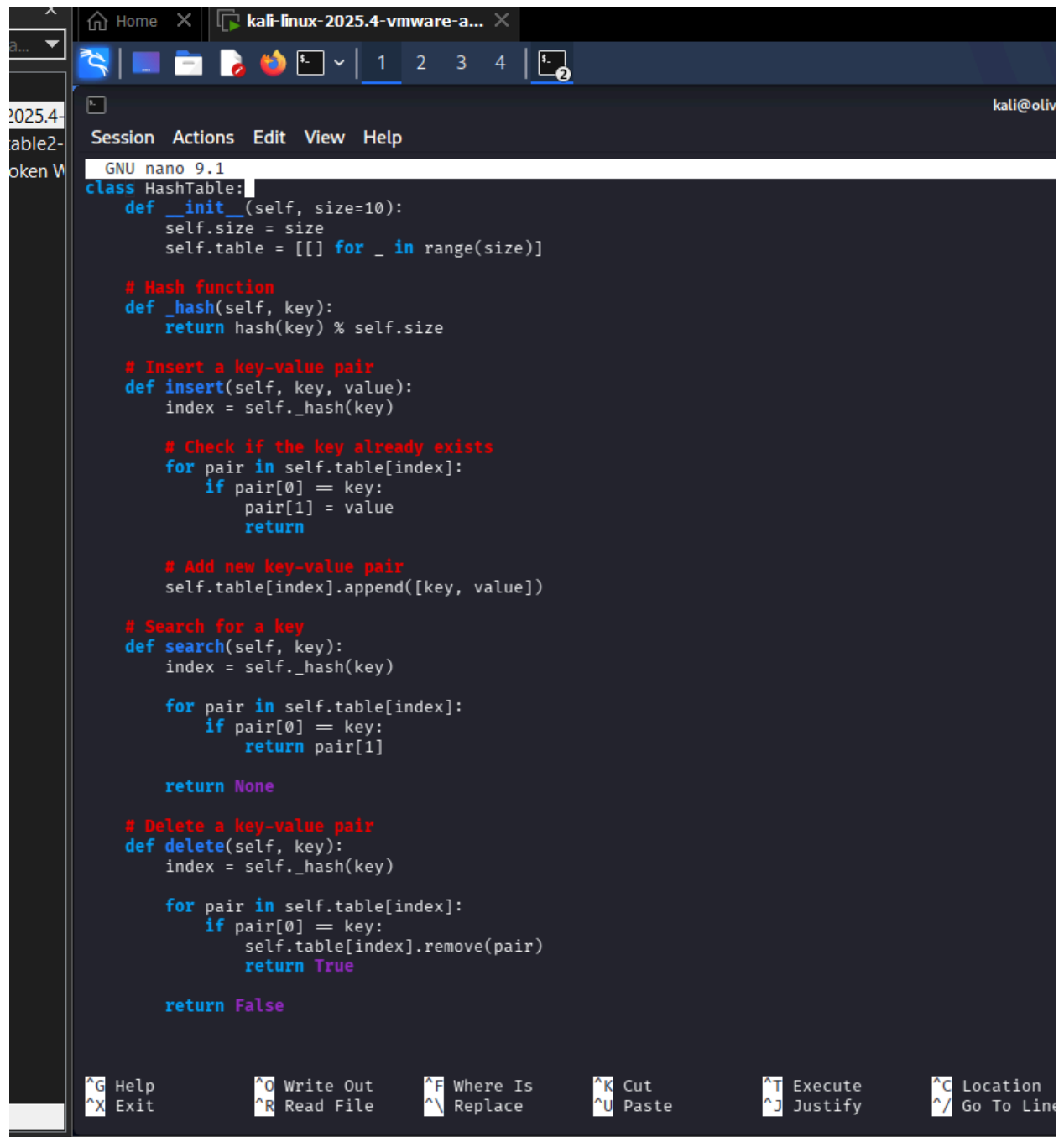
```
# Confirm that the deleted key is no longer available
print("Search language after deletion:", ht.search("language"))
```

```
# Display the hash table
print("\nHash Table:")
for index, bucket in enumerate(ht.table):
    print(index, bucket)
```

Ctrl + O → Enter → Ctrl + X

Command used:

```
python3 hash_table.py
```



The image shows a Kali Linux terminal window with a nano editor open. The editor displays a Python class named `HashTable` with methods for initialization, hashing, insertion, searching, and deletion. The terminal window has a dark theme and a menu bar at the top with options like Session, Actions, Edit, View, and Help. The bottom of the window features a status bar with various keyboard shortcuts.

```
GNU nano 9.1
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    # Hash function
    def _hash(self, key):
        return hash(key) % self.size

    # Insert a key-value pair
    def insert(self, key, value):
        index = self._hash(key)

        # Check if the key already exists
        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return

        # Add new key-value pair
        self.table[index].append([key, value])

    # Search for a key
    def search(self, key):
        index = self._hash(key)

        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]

        return None

    # Delete a key-value pair
    def delete(self, key):
        index = self._hash(key)

        for pair in self.table[index]:
            if pair[0] == key:
                self.table[index].remove(pair)
                return True

        return False
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line